

UNIVERSIDAD AUTÓNOMA “JUAN MISAE SARACHO”

**FACULTAD DE CIENCIAS Y TECNOLOGÍA
DIRECCIÓN DE POSGRADO
CARRERA DE INGENIERÍA INFORMÁTICA**



TRABAJO FINAL DE DIPLOMADO

MAX PIZZAPP: SISTEMA WEB Y MÓVIL DE GESTIÓN DE PEDIDOS PARA RECEPCIÓN Y COCINA DE MAX'S PIZZAS EN VILLA FÁTIMA

Trabajo Final presentado para la obtención del título de
DIPLOMADO EN DESARROLLO WEB Y APLICACIONES MÓVILES

Postulante: Jorge Luis Gutierrez Rivera

Tutor: M.Sc. Ing. Isaac Lange Aguilar

Tarija – Bolivia

2026

UNIVERSIDAD AUTÓNOMA “JUAN MISAE SARACHO”

**FACULTAD DE CIENCIAS Y TECNOLOGÍA
DIRECCIÓN DE POSGRADO
CARRERA DE INGENIERÍA INFORMÁTICA**



TRABAJO FINAL DE DIPLOMADO

MAX PIZZAPP: SISTEMA WEB Y MÓVIL DE GESTIÓN DE PEDIDOS PARA RECEPCIÓN Y COCINA DE MAX'S PIZZAS EN VILLA FÁTIMA

Trabajo Final presentado para la obtención del título de
DIPLOMADO EN DESARROLLO WEB Y APLICACIONES MÓVILES

Postulante: Jorge Luis Gutierrez Rivera

Tutor: M.Sc. Ing. Isaac Lange Aguilar

Tarija – Bolivia

2026

HOJA DE APROBACIÓN

MAX PIZZAPP: SISTEMA WEB Y MÓVIL DE GESTIÓN DE PEDIDOS PARA
RECEPCIÓN Y COCINA DE MAX'S PIZZAS EN VILLA FÁTIMA

Postulante: Jorge Luis Gutierrez Rivera

Lugar y fecha de aprobación: _____

Tutor

Tribunal 1

Tribunal 2

HOJA DE ADVERTENCIA

El Tribunal Calificador del presente trabajo de diplomado no se solidariza ni responsabiliza con la forma, términos, modos y expresiones vertidas en el mismo, siendo esta responsabilidad del autor.

DEDICATORIA

“A los pilares fundamentales de mi vida, sin los cuales no podría seguir adelante: a mi madre, que nunca renunció ni perdió la esperanza en mí; a mis hermanos, que siempre son el aliento que necesito para continuar; a mis tutores, por el apoyo y el conocimiento; y a quien hubiera querido que estuviera aquí, sintiéndose orgulloso de lo que soy hoy y de esta meta que estoy logrando: a mi papá, que está en el cielo.”

AGRADECIMIENTOS

Quiero expresar mi profundo agradecimiento a los docentes del Diplomado por compartir sus conocimientos, fomentar el espíritu de investigación y fortalecer nuestra formación mediante los nuevos conocimientos adquiridos. A mis compañeros del Diplomado, por compartir conocimientos y experiencias que enriquecieron mi aprendizaje. A mi familia, por su paciencia y apoyo incondicional, y a mis amigos, por ser un sostén emocional en los momentos más desafiantes. Este trabajo refleja el esfuerzo conjunto de todos ellos.

ÍNDICE DE CONTENIDO

HOJA DE APROBACIÓN	iii
HOJA DE ADVERTENCIA.....	iv
DEDICATORIA	v
AGRADECIMIENTOS.....	vi
ÍNDICE DE CONTENIDO	vii
ÍNDICE DE TABLAS	xi
ÍNDICE DE FIGURAS.....	xiv
ÍNDICE DE ANEXOS.....	xvii
RESUMEN.....	xviii
CAPÍTULO 1. PLANTEAMIENTO DEL PROBLEMA	2
1.1 Antecedentes	2
1.2 Descripción del problema	4
1.3 Planteamiento del problema	5
1.4 Justificación del problema	6
1.5 Objetivos	6
1.5.1 Objetivo general.....	6
1.5.2 Objetivos específicos	7
CAPÍTULO 2. DESARROLLO DE LA SOLUCIÓN	9
2.1 Fundamentos conceptuales y tecnológicos	9
2.1.1 Arquitectura cliente-servidor y organización modular	9
2.1.2 Servicios web y comunicación mediante HTTP.....	9
2.1.3 Persistencia relacional, integridad y transacciones	9
2.1.4 Comunicación en tiempo real y eventos	10
2.1.5 Autenticación y autorización	10
2.1.6 Desarrollo multiplataforma e interfaz adaptable	10
2.1.7 Contenedores y despliegue reproducible	10
2.2 Metodología de desarrollo	10
2.2.1 Uso de asistentes de inteligencia artificial.....	12

2.2.2 Evidencia de aplicación de la metodología	13
2.3 Análisis de requisitos.....	13
2.3.1 Actores y perfiles de usuario.....	13
2.3.2 Requisitos funcionales	14
2.3.3 Requisitos no funcionales	15
2.3.4 Reglas de negocio	16
2.3.5 Casos de uso.....	16
CU-01 Iniciar sesión	17
CU-02 Registrar pedido	17
CU-03 Actualizar el estado del pedido	17
2.4 Diseño de la solución	17
2.4.1 Arquitectura del sistema	18
2.4.2 Modelo de datos	18
2.4.3 Diseño de la interfaz	19
2.4.4 Diseño de la interfaz de programación de aplicaciones	20
2.5 Justificación del stack tecnológico.....	21
2.6 Implementación.....	22
2.6.1 Organización del código	22
2.6.2 Avance por tarjeta.....	22
2.6.3 El flujo implementado: del mostrador a la cocina	23
2.6.4 Decisiones de implementación	23
2.6.5 Interfaz implementada	23
2.6.6 Fragmentos de código ilustrativos.....	26
2.6.7 Dificultades y su resolución.....	26
2.7 Seguridad.....	26
2.7.1 Autenticación y sesiones	26
2.7.2 Autorización por rol	27
2.7.3 Entradas y exposición.....	27
2.7.4 Cifrado y secretos	27
2.8 Pruebas.....	28

2.8.1 Plan de pruebas.....	28
2.8.2 Pruebas automatizadas	29
2.8.3 Casos de prueba	29
2.8.4 Requisitos no funcionales	29
2.8.5 Alcance y límites de las pruebas.....	31
2.9 Despliegue	31
2.9.1 Plataforma y servicios.....	31
2.9.2 Procedimiento y verificación	32
2.9.3 Entornos y distribución.....	32
2.9.4 Monitoreo y limitaciones	33
CAPÍTULO 3. CONCLUSIONES Y RECOMENDACIONES.....	35
3.1 Conclusiones.....	35
3.2 Recomendaciones.....	36
BIBLIOGRAFÍA CONSULTADA.....	37
ANEXOS	41
ANEXO A. MANUAL DE USUARIO	42
A.1 Acceso al sistema	42
A.2 Recepción	43
A.3 Cocina	46
A.4 Disponibilidad por perfil	49
A.5 Mensajes frecuentes	50
A.6 Reglas importantes	51
ANEXO B. MANUAL DE INSTALACIÓN Y DESPLIEGUE	52
Parte 1. Instalación local.....	52
B.1 Requisitos	52
B.2 Preparar Windows e instalar Docker Desktop	53
B.3 Descargar Max Pizzapp	56
B.4 Instalar el sistema	57
B.5 Comprobar el funcionamiento.....	59
B.6 Detener y volver a abrir	61

B.7 Problemas frecuentes.....	62
B.8 Validación externa del procedimiento	63
Parte 2. Despliegue de producción	64
B.9 Requisitos del servidor	64
B.10 Primer despliegue	64
B.11 Verificar producción.....	65
B.12 Actualizar y distribuir el APK.....	66
ANEXO C. ENLACE AL REPOSITORIO DE CÓDIGO.....	67
ANEXO D. ACCESO PÚBLICO O INSTALABLE	68
ANEXO E. DIAGRAMAS EN TAMAÑO LEGIBLE	69
ANEXO F. DOCUMENTACIÓN COMPLEMENTARIA.....	73
F.1 Reglas de negocio.....	73
F.2 Componentes de la arquitectura	73
F.3 Diccionario de cliente	74
F.4 Diccionario de pedido	74
F.5 Diccionario de detalle_pedido.....	75
F.6 Diccionario de producto	76
F.7 Diccionario de historial de estado	76
F.8 Tarjetas del desarrollo	77
F.9 Matriz OWASP Top 10:2025.....	78
F.10 Configuración del canal en vivo	78
F.11 Matriz completa de casos de prueba	79
F.12 Evidencia complementaria.....	82

ÍNDICE DE TABLAS

<i>Tabla 1. Soluciones informáticas existentes en el dominio. Fuente: elaboración propia con base en Fudo (s. f.), Loyverse (s. f.) y Open Source Point of Sale (s. f.).</i>	3
<i>Tabla 2. Cronograma efectivamente ejecutado al 8 de octubre de 2026. Fuente: elaboración propia.</i>	11
<i>Tabla 3. Actores del sistema. Fuente: elaboración propia con base en los requisitos vigentes.</i>	14
<i>Tabla 4. Requisitos funcionales, criterios de aceptación y prioridad. Fuente: elaboración propia con base en el backlog vigente.</i>	15
<i>Tabla 5. Requisitos no funcionales y criterios de verificación. Fuente: elaboración propia.</i>	16
<i>Tabla 6. Contrato vigente de la API. Fuente: elaboración propia a partir de docs/api/openapi.yaml.</i>	21
<i>Tabla 7. Stack tecnológico y criterios de selección. Fuente: elaboración propia con base en la configuración vigente del proyecto.</i>	22
<i>Tabla 8. Validación doble de los datos principales. Fuente: implementación y casos de prueba al 07/10/2026.</i>	28
<i>Tabla 9. Niveles de prueba al 8 de octubre de 2026. Fuente: elaboración propia a partir de docs/pruebas/README.md.</i>	29
<i>Tabla 10. Síntesis de casos de prueba al 8 de octubre de 2026. Fuente: elaboración propia a partir de los reportes versionados.</i>	29
<i>Tabla 11. Propagación de eventos en producción, 07/10/2026. Fuente: docs/pruebas/reportes/tiempo-real.txt.</i>	30
<i>Tabla 12. Verificación externa del despliegue. Fuente: comprobaciones del autor y reportes de producción, 05–08/10/2026.</i>	32
<i>Tabla 13. Entornos de ejecución. Fuente: configuración vigente del proyecto.</i>	32

Tabla 14. Grado de cumplimiento de los objetivos específicos. Fuente: elaboración propia.	35
Tabla A.1. Responsabilidad sobre la disponibilidad. Fuente: reglas implementadas de RF-13 y D-76.	49
Tabla A.2. Mensajes de la aplicación y acción recomendada. Fuente: textos de Max Pizzapp 0.4.0.	50
Tabla B.1. Comprobaciones previas. Fuente: procedimiento validado el 08/10/2026.	52
Tabla B.2. Incidencias y solución documentada. Fuente: manual corregido tras la prueba externa.	62
Tabla B.3. Comprobaciones externas del despliegue. Fuente: README y reportes de producción.	65
<i>Tabla D.1. Accesos públicos e instalables. Fuente: verificación del autor, 07/10/2026.</i>	<i>68</i>
<i>Tabla F.1. Reglas de negocio vigentes. Fuente: decisiones y esquema del proyecto.</i>	<i>73</i>
<i>Tabla F.2. Componentes y responsabilidades. Fuente: configuración vigente.</i>	<i>74</i>
<i>Tabla F.3. Diccionario de datos de cliente. Fuente: esquema vigente.</i>	<i>74</i>
<i>Tabla F.4. Diccionario de datos de pedido. Fuente: esquema vigente.</i>	<i>75</i>
<i>Tabla F.5. Diccionario de datos de detalle_pedido. Fuente: esquema vigente.</i>	<i>76</i>
Tabla F.6. Diccionario de datos de producto. Fuente: esquema vigente.	76
Tabla F.7. Diccionario de datos de historial_estado. Fuente: esquema vigente.	77
Tabla F.8. Tarjetas e incrementos al 8 de octubre de 2026. Fuente: tablero, evidencias y planes del proyecto.	77
Tabla F.9. Riesgos OWASP Top 10:2025 y tratamiento en Max Pizzapp. Fuente: OWASP Foundation (2025) y controles verificados.	78

Tabla F.10. Matriz completa CP-01 a CP-31. Fuente: docs/pruebas/README.md y reportes vigentes al 08/10/2026.	81
--	----

ÍNDICE DE FIGURAS

Figura 1. Estado del tablero Kanban al cierre documental, 10 de octubre de 2026. Fuente: captura propia de GitHub Projects.	12
<i>Figura 2. Diagrama de casos de uso de Max Pizzapp. Fuente: elaboración propia con base en los requisitos vigentes.</i>	<i>16</i>
<i>Figura 3. Arquitectura actual de Max Pizzapp. Fuente: elaboración propia con base en la configuración vigente del proyecto.</i>	<i>18</i>
<i>Figura 4. Modelo entidad-relación vigente. Fuente: elaboración propia a partir del esquema y sus migraciones.</i>	<i>19</i>
<i>Figura 5. Prototipos de las pantallas principales y su adaptación. Fuente: elaboración propia a partir de la interfaz vigente.</i>	<i>20</i>
Figura 6. Formulario de venta de Recepción con pizzas y bebidas reales. Fuente: captura del sistema desplegado, 07/10/2026.	24
Figura 7. APK 0.4.0 de Cocina con un pedido que incluye una bebida. Fuente: captura del sistema desplegado, 07/10/2026.	25
<i>Figura 8. Las transiciones de estado y el rol que hace cada una. Fuente: backend/src/pedidos/estados.js.</i>	<i>26</i>
Figura 9. La venta, los pedidos de Recepción y la cola de Cocina a 768 × 1024 px, en Edge 154. Fuente: captura del sistema desplegado, 02/10/2026.	30
Figura 10. Monitor de salud de la API al 10 de octubre de 2026. Fuente: UptimeRobot.	33
Figura A.1. Pantalla de acceso con la identidad visual vigente. Fuente: captura del sistema desplegado, 08/10/2026.	43
Figura A.2. Venta preparada con pizza, bebidas y total. Fuente: captura del sistema desplegado, 07/10/2026.	44
Figura A.3. Selección de una pizza de dos mitades. Fuente: captura del sistema desplegado, 25/09/2026.	44
Figura A.4. Venta directa de bebidas. Fuente: captura del sistema desplegado, 07/10/2026.	45

Figura A.5. Incorporación de una pizza a un pedido enviado. Fuente: captura del sistema desplegado, 08/10/2026.	46
Figura A.6. Aviso de pedido listo en Recepción. Fuente: captura del sistema desplegado, 08/10/2026.	46
Figura A.7. Pedido nuevo en el APK de Cocina, segundo dispositivo. Fuente: captura del sistema desplegado, 08/10/2026.	48
Figura A.8. Disponibilidad de pizzas administrada desde Cocina. Fuente: captura del sistema desplegado, 08/10/2026.	49
Figura A.9. Aviso en Recepción cuando Cocina agota todas las pizzas. Fuente: captura del sistema desplegado, 08/10/2026.	50
Figura B.1. Verificación de Windows 11 Home 25H2 en el equipo de prueba. Fuente: fotografía de la prueba externa, 08/10/2026.	53
Figura B.2. Incidencia de Virtual Machine Platform detectada en la prueba. Fuente: fotografía de la prueba externa, 08/10/2026.	54
Figura B.3. Instalación de WSL aplicada durante la prueba. Fuente: fotografía de la prueba externa, 08/10/2026.	55
Figura B.4. Docker Desktop con el motor en ejecución. Fuente: fotografía de la prueba externa, 08/10/2026.	56
Figura B.5. Descarga del archivo ZIP del proyecto. Fuente: fotografía de la prueba externa, 08/10/2026.	57
Figura B.6. Construcción de los componentes durante el paso [4/6]. Fuente: fotografía de la prueba externa, 08/10/2026.	58
Figura B.7. Aviso del Firewall de Windows observado en la prueba. Fuente: fotografía de la prueba externa, 08/10/2026.	59
Figura B.8. Instalación completada en el paso [6/6], sin exponer la contraseña. Fuente: fotografía de la prueba externa, 08/10/2026.	59
Figura B.9. Pedido de prueba recibido en Cocina. Fuente: fotografía de la prueba externa, 08/10/2026.	60
Figura B.10. Pedido listo para entregar en Recepción. Fuente: fotografía de la prueba externa, 08/10/2026.	61

Figura B.11. Contenedores de Max Pizzapp en funcionamiento. Fuente: fotografía de la prueba externa, 08/10/2026.	62
Figura B.12. Encargado de recepción durante la prueba externa. Fuente: fotografía autorizada, 08/10/2026.	64
<i>Figura E.1. Diagrama de casos de uso. Fuente: elaboración propia.</i>	<i>69</i>
<i>Figura E.2. Arquitectura actual. Fuente: elaboración propia.</i>	<i>70</i>
<i>Figura E.3. Modelo entidad-relación. Fuente: elaboración propia.</i>	<i>71</i>
<i>Figura E.4. Prototipos de las pantallas. Fuente: elaboración propia.</i>	<i>72</i>
<i>Figura F.1. Configuración resumida del latido y del límite de sesión. Fuente: backend/src/tiempo-real.js.</i>	<i>78</i>
Figura F.2. Ejecución de comprobaciones de seguridad contra producción. Fuente: captura propia.	82
Figura F.3. Respuesta observada después del vencimiento del token. Fuente: captura propia.	82
Figura F.4. Versión de Chrome identificada para CP-27. Fuente: captura propia.	82
Figura F.5. Página pública de estado del monitoreo. Fuente: UptimeRobot, 10/10/2026.	83

ÍNDICE DE ANEXOS

ANEXO A. MANUAL DE USUARIO	42
ANEXO B. MANUAL DE INSTALACIÓN Y DESPLIEGUE.....	52
ANEXO C. ENLACE AL REPOSITORIO DE CÓDIGO	67
ANEXO D. ACCESO PÚBLICO O INSTALABLE	68
ANEXO E. DIAGRAMAS EN TAMAÑO LEGIBLE.....	69
ANEXO F. DOCUMENTACIÓN COMPLEMENTARIA	73

RESUMEN

En el local central de Max's Pizzas, en Villa Fátima, Recepción registraba los pedidos en papel y los comunicaba de palabra a Cocina, sin una referencia compartida de su estado. La responsable estimó entre seis y siete pedidos demorados y entre dos y tres pedidos mal preparados, traspapelados u olvidados por día; estas cifras expresan su apreciación y no mediciones del sistema. El trabajo desarrolló Max Pizzapp, una solución web adaptable y Android para gestionar el flujo entre Recepción y Cocina con trazabilidad y sincronización en tiempo real. Se aplicó Kanban y una arquitectura con Flutter, Node.js y Express, PostgreSQL, Keycloak, Socket.IO, Docker Compose y Caddy. Se implementaron los siete requisitos Must y los cinco Should, incluido RF-13 para administrar la disponibilidad por categoría y propagar sus cambios. La API aprobó 332 de 332 pruebas y la aplicación 338 de 338; también se ejecutaron seis sondas en producción y 25 comprobaciones de error. Los 31 casos de prueba fueron aprobados. Las mediciones de cuatro eventos registraron un máximo global de 450 ms, inferior al límite de 2 s, y la prueba de carga alcanzó un percentil 95 de 296,36 ms. El sistema está desplegado públicamente y el encargado de recepción completó una instalación local y el flujo de venta a entrega siguiendo el manual, con tres intervenciones del autor que originaron ajustes documentales. No se atribuyen reducciones de demoras o errores sin una evaluación posterior de uso real.

CAPÍTULO 1

CAPÍTULO 1. PLANTEAMIENTO DEL PROBLEMA

Max Pizzapp se desarrolló para apoyar la gestión de pedidos de Max's Pizzas en el local central de Villa Fátima, Tarija. El estudio se concentra en el intercambio de información entre dos perfiles operativos: Recepción, que registra y entrega los pedidos, y Cocina, que organiza la preparación y comunica su avance. Las otras dos sucursales del negocio, ubicadas en Villa Avaroa y Tabladita, no forman parte del piloto.

Este capítulo parte del contexto tecnológico de la gestión gastronómica y contrasta soluciones existentes con las necesidades del negocio. Luego describe el proceso manual, formula el problema y justifica el alcance del proyecto. La secuencia concluye con los objetivos que orientan el desarrollo del trabajo, sin atribuir al sistema resultados operativos que todavía no hayan sido medidos.

1.1 Antecedentes

En un establecimiento gastronómico, el pedido enlaza la atención al cliente con la preparación en cocina. Tan y Chang (2010) describen el proceso de servicio como una secuencia en la que el personal registra la orden y la transmite a cocina; en el sistema estudiado, esa información se digitaliza y se envía de forma inmediata al área de preparación. Este antecedente sitúa la gestión del pedido como un problema de continuidad de la información entre quienes toman la orden y quienes la preparan, aunque cada establecimiento deba adaptar el flujo a su propia operación.

Las pantallas de cocina constituyen una forma documentada de apoyar esa continuidad. Oracle (s. f.) define un sistema de pantalla de cocina como un dispositivo que comunica la información del pedido al área de preparación y la muestra en monitores para que el personal conozca qué debe preparar; su documentación también contempla estados, temporizadores y detalles de

cada orden. Estas funciones permiten presentar la comanda y su avance dentro del flujo de cocina, sin suponer que una misma configuración responda a las reglas de todos los negocios.

En este dominio conviven plataformas de gestión gastronómica, pantallas de cocina y puntos de venta de código abierto. La comparación permite reconocer capacidades generales y precisar cuáles requieren adaptación al proceso del piloto.

La Tabla 1 contrasta tres soluciones representativas según su documentación pública, sin atribuirles carencias que las fuentes no demuestren. El análisis se limita al envío del pedido a cocina, el seguimiento de la preparación y el ajuste al flujo de Max's Pizzas.

Solución	Alcance	Qué resuelve	Qué deja sin resolver	Fuente
Fudo	Plataforma gastronómica comercial con ventas, comandas, cocina, inventario, caja, reportes y reparto.	Comandas digitales, tiempos, alertas y aviso cuando la orden está lista.	Incluye procesos ajenos al piloto y planes comerciales. Su documentación no describe las reglas particulares de venta, adición y cancelación de Max's Pizzas.	Fudo (s. f.)
Loyverse KDS	Pantalla de cocina para iPad y tabletas Android conectada a Loyverse POS.	Muestra artículos, cantidades, modificadores, notas y espera; permite marcar artículos o pedidos y emite alertas.	Depende de Loyverse POS y de una app para tableta. La documentación no identifica una web adaptada a los dos roles y reglas del piloto.	Loyverse (s. f.)
Open Source Point of Sale (OSPOS)	Punto de venta web de código abierto; su versión estable publicada es 3.4.1.	Ventas, inventario, clientes, proveedores, registros, reportes y permisos.	Su documentación no identifica pantalla de cocina ni un flujo Recepción-Cocina configurado para el piloto.	Open Source Point of Sale (s. f.)

Tabla 1. Soluciones informáticas existentes en el dominio. Fuente: elaboración propia con base en Fudo (s. f.), Loyverse (s. f.) y Open Source Point of Sale (s. f.).

Fudo y Loyverse KDS documentan el envío y seguimiento de pedidos en cocina. OSPOS ofrece ventas, inventario, registros y permisos, pero su documentación pública no identifica una pantalla de cocina con intercambio de estados entre los dos puestos del piloto.

Max Pizzapp aborda la necesidad de representar el proceso del local central de Villa Fátima mediante dos perfiles y un pedido compartido. El piloto no sustituye caja, inventario, reparto, promociones condicionadas ni operación multisede.

1.2 Descripción del problema

Max's Pizzas opera tres locales en Tarija. El piloto corresponde al local central de Villa Fátima, con ocho mesas y pedidos por mensajería y reparto externo, cuya logística queda fuera del sistema. Durante la entrevista del 18 de septiembre de 2026, la responsable informó un horario de atención de 18:00 a 23:30.

En el proceso previo al sistema, Recepción anotaba el pedido en papel y lo pasaba de palabra a Cocina. La preparación avanzaba con la información que el personal alcanzaba a escuchar o recibir, sin una vista compartida que indicara qué pedidos estaban pendientes, en preparación o listos. Por ello, Recepción debía consultar directamente a Cocina para responder al cliente y no contaba con un registro común desde el cual seguir cada cambio.

La entrevista a Herlinda Duran, dueña de Max's Pizzas, del 18 de septiembre de 2026 permitió caracterizar el proceso. Los valores son estimaciones de la responsable sobre la semana previa, no mediciones instrumentadas: entre 19:30 y 22:00 se atendían de 50 a 70 pedidos, y el tiempo habitual de preparación fue estimado entre 12 y 15 minutos.

Sin un estado compartido, Recepción podía advertir una demora recién ante la consulta del cliente. La responsable estimó entre seis y siete pedidos demorados por día en la semana de referencia.

La falta de un registro común de disponibilidad podía detectarse al iniciar la preparación. Contactar al cliente, sustituir el producto y reorganizar el trabajo añadía, según la responsable, entre 10 y 15 minutos.

El traspaso verbal y los papeles separados también dificultaban reconstruir el pedido. La responsable estimó entre dos y tres pedidos diarios mal preparados, traspapelados u olvidados (entrevista del 18 de septiembre de 2026).

Estas estimaciones describen un problema de coordinación y trazabilidad; no constituyen una línea base automática ni permiten inferir causas estadísticas. Cualquier evaluación del efecto del sistema requiere mediciones posteriores.

1.3 Planteamiento del problema

El local central de Max's Pizzas no disponía de un medio compartido que permitiera a Recepción registrar los pedidos y a Cocina comunicar su avance sobre una misma fuente de información. En la semana previa a la entrevista del 18 de septiembre de 2026, la responsable estimó entre seis y siete pedidos demorados y entre dos y tres pedidos mal preparados, traspapelados u olvidados por día. Estas cifras caracterizan el proceso manual y no constituyen resultados medidos por Max Pizzapp. La necesidad central consiste en reducir la dependencia de consultas verbales y mantener una referencia común sobre el estado de cada pedido. A partir de este diagnóstico se formula la siguiente pregunta:

¿Cómo mejorar la gestión y trazabilidad de los pedidos entre Recepción y Cocina del local central de Max's Pizzas, reduciendo la dependencia de la

coordinación verbal y permitiendo conocer oportunamente el estado de cada pedido?

1.4 Justificación del problema

La factibilidad técnica del proyecto se sustenta en la disponibilidad de tecnologías web, herramientas de desarrollo e infraestructura para publicar la solución. También se cuenta con acceso al proceso del negocio y con equipos de escritorio o tabletas adecuados para los puestos considerados. Al limitar el piloto al local central de Villa Fátima y a los perfiles de Recepción y Cocina, el desarrollo y la verificación pueden concentrarse en el recorrido principal del pedido dentro de un alcance viable.

La propuesta es pertinente porque responde directamente a la dificultad observada en el intercambio de información entre Recepción y Cocina. El sistema se concentra en el registro y el seguimiento compartido del pedido entre esos dos perfiles, en correspondencia con el problema diagnosticado. Este alcance acotado evita incorporar módulos ajenos al flujo central del piloto.

El impacto esperado consiste en disponer de una referencia compartida del pedido, conservar la trazabilidad de sus cambios y diferenciar las acciones que corresponden a Recepción y Cocina. Estas capacidades pueden verificarse técnicamente y podrían disminuir la necesidad de consultas verbales para conocer el avance del pedido y responder al cliente. Sin embargo, todavía no existe evidencia suficiente para afirmar una reducción comprobada de demoras, errores o tiempos de atención; ese efecto deberá evaluarse mediante datos obtenidos durante el uso operativo y comparados con el proceso previo.

1.5 Objetivos

1.5.1 Objetivo general

Desarrollar un sistema web y móvil, con interfaz adaptable, para gestionar los pedidos entre los perfiles de Recepción y Cocina del local central de Max's Pizzas, con trazabilidad de estados y sincronización de la información en tiempo real.

1.5.2 Objetivos específicos

1. Especificar los requisitos del proceso de toma y preparación de pedidos de Max's Pizzas mediante 14 requisitos funcionales con criterios de aceptación y 5 requisitos no funcionales con métricas verificables, priorizados con MoSCoW.
2. Definir la arquitectura de la solución y el modelo de datos relacional de cinco entidades de Max Pizzapp, con su diccionario de datos y el contrato de la API.
3. Implementar los módulos de Recepción y Cocina con sincronización en tiempo real del estado y los cambios del pedido.
4. Aplicar mecanismos de autenticación, autorización por rol y protección de las comunicaciones de Max Pizzapp.
5. Ejecutar pruebas funcionales, de integración y de rendimiento que permitan verificar el flujo de pedidos y la sincronización en tiempo real.
6. Desplegar Max Pizzapp en una dirección pública accesible y documentar el procedimiento de despliegue.

CAPÍTULO 2

CAPÍTULO 2. DESARROLLO DE LA SOLUCIÓN

El presente capítulo desarrolla los fundamentos y decisiones técnicas que sustentan Max Pizzapp. Se parte de los conceptos necesarios para comprender su arquitectura y tecnologías y, posteriormente, se describe la metodología aplicada, los requisitos, el diseño, la selección del stack, la implementación, los mecanismos de seguridad, las pruebas y el despliegue de la solución.

2.1 Fundamentos conceptuales y tecnológicos

2.1.1 Arquitectura cliente-servidor y organización modular

La arquitectura cliente-servidor separa la presentación, la lógica del dominio y la persistencia. En Max Pizzapp, los clientes web y Android solicitan operaciones a una API que aplica las reglas antes de acceder a la base; esta separación evita que cada interfaz replique validaciones críticas y mantiene a PostgreSQL fuera del acceso directo (Bass, Clements y Kazman, 2021).

2.1.2 Servicios web y comunicación mediante HTTP

HTTP organiza las solicitudes y respuestas mediante métodos, rutas y códigos de estado. GET consulta, POST crea o procesa y PATCH aplica cambios parciales; OpenAPI 3.0.3 documenta ese contrato de forma independiente del lenguaje (Fielding, Nottingham y Reschke, 2022; OpenAPI Initiative, 2020). Max Pizzapp usa este modelo para pedidos, carta, estados y disponibilidad.

2.1.3 Persistencia relacional, integridad y transacciones

El modelo relacional organiza datos en tablas vinculadas por claves y restricciones (Codd, 1970). Las transacciones hacen atómicas las operaciones compuestas: un pedido, sus detalles y su historial se confirman juntos o se revierten. Esta propiedad sostiene el cálculo de precios y evita estados parciales cuando una operación falla.

2.1.4 Comunicación en tiempo real y eventos

WebSocket mantiene un canal bidireccional; Socket.IO añade eventos, salas y reconexión sobre ese canal (Fette y Melnikov, 2011; Socket.IO, 2026a). Los eventos de Max Pizzapp notifican pedidos, estados, agregados y disponibilidad, mientras la API y la base siguen siendo la fuente de verdad después de una reconexión.

2.1.5 Autenticación y autorización

La autenticación identifica a la persona usuaria y la autorización delimita sus acciones. OpenID Connect con código de autorización y PKCE evita que la aplicación maneje la contraseña y protege el canje del código (Sakimura et al., 2023; Sakimura, Bradley y Agarwal, 2015). Keycloak emite los tokens; la API verifica firma, vigencia, emisor, audiencia y rol.

2.1.6 Desarrollo multiplataforma e interfaz adaptable

Flutter permite compartir lógica y componentes entre web y Android, sin exigir la misma composición visual (Flutter, 2026a; 2026b). Recepción utiliza una vista amplia para registrar y seguir pedidos; Cocina usa una cola de tarjetas en el APK o el navegador. La adaptación responde a tareas distintas, no solo al tamaño de pantalla.

2.1.7 Contenedores y despliegue reproducible

Un contenedor ejecuta un servicio con dependencias definidas; Docker Compose declara servicios, redes, volúmenes y variables como una aplicación conjunta (Docker, s. f.a; s. f.b). Max Pizzapp separa proxy, API, identidad y base de datos, lo que permite repetir la instalación sin convertir el servidor en una configuración manual irreproducible.

2.2 Metodología de desarrollo

Max Pizzapp se desarrolló con Kanban y un tablero de GitHub Projects organizado en Por hacer, En curso, En pruebas y Hecho. El trabajo individual

mantuvo WIP = 1 para terminar cada tarjeta antes de iniciar la siguiente. No se aplicó Scrum porque no se ejecutaron sus roles ni ceremonias de equipo.

Cada tarjeta siguió un flujo continuo: plan aprobado, implementación, pruebas, revisión y cierre con evidencia. El tablero, los planes, la bitácora de decisiones, el registro de errores y las pruebas permiten reconstruir la evolución del proyecto.

La Tabla 2 resume por semana los incrementos y tarjetas cerrados durante el calendario académico.

Semana	Periodo	Contenido ejecutado o previsto	Observación	Estado
Semana 1	11-20 sep	Incremento 0: alcance, requisitos, arquitectura, modelo, repositorio y E1; RF-01 a RF-14 y RNF-01 a RNF-05 como especificación.	Inicio y delimitación.	Completada
Semana 2	21-27 sep	Incrementos 1 y 2, tarjetas 01-06: persistencia, HTTPS, identidad, API, carta y flujo Recepción-Cocina-Recepción; RF-01 a RF-09, RF-11 y RF-14.	Los pedidos previstos para la Semana 3 se cerraron en la tarjeta 06.	Completada
Semana 3	28 sep-3 oct	Incrementos 3 y 4: canal y sesión (07), APK (12), seguridad (09) y pruebas (10); RF-03, RF-04, RF-06 a RF-08 y RNF-01 a RNF-05.	RF-13 (tarjeta 08) pasó a la Semana 4.	Completada
Semana 4	5-10 oct	Cierre E4: disponibilidad, bebidas reales, estabilización, pruebas, despliegue y documentación.	RF-13, identidad visual y prueba externa se completaron el 8 de octubre; continuó el cierre documental.	En cierre

Tabla 2. Cronograma efectivamente ejecutado al 8 de octubre de 2026. Fuente: elaboración propia.

La Figura 1 muestra el estado del tablero al 10 de octubre de 2026: 26 tarjetas se encontraban en Hecho, una en curso, ninguna en pruebas y dos permanecían en Por hacer. Las dos tarjetas pendientes corresponden a RF-10 y RF-12, priorizadas como Could, mientras que la tarjeta en curso corresponde a los manuales y al cierre documental.

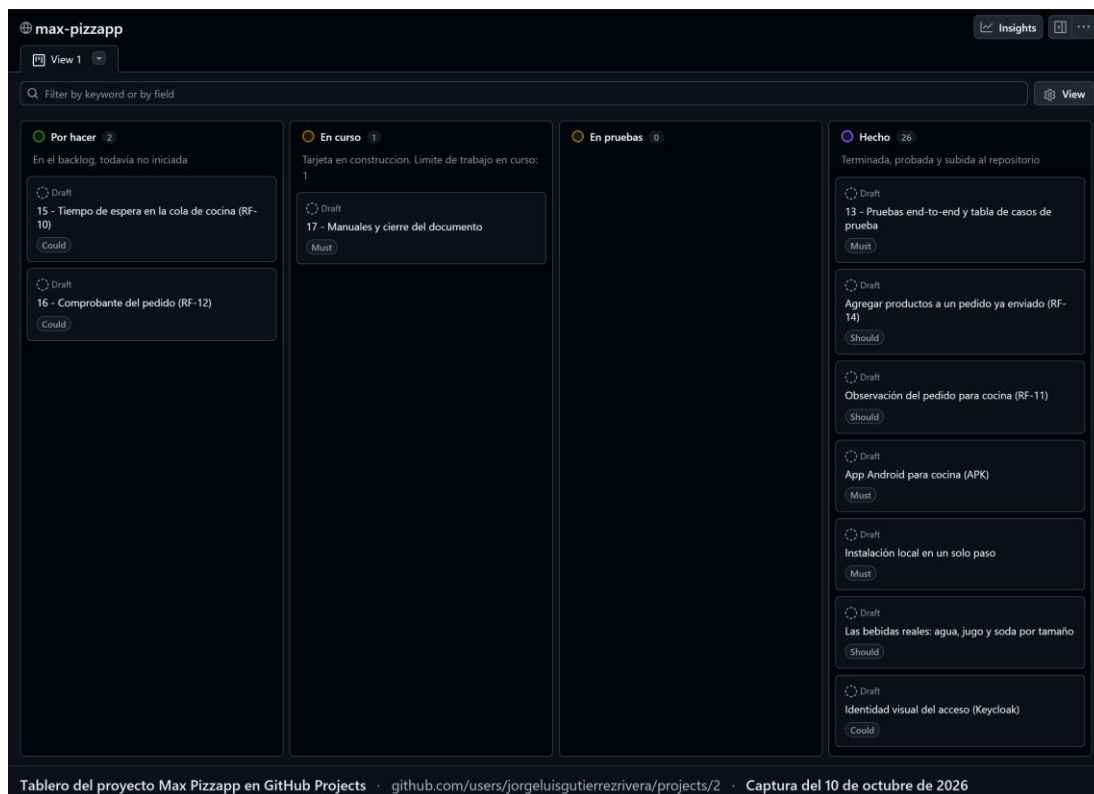


Figura 1. Estado del tablero Kanban al cierre documental, 10 de octubre de 2026. Fuente: captura propia de GitHub Projects.

2.2.1 Uso de asistentes de inteligencia artificial

Durante el desarrollo se utilizaron asistentes de inteligencia artificial como herramientas de apoyo. Claude Code, de Anthropic, se empleó para proponer borradores de código y documentación técnica, apoyar la revisión y depuración de implementaciones, explorar alternativas, revisar configuraciones y generar casos de prueba. ChatGPT, de OpenAI, se utilizó para revisar la coherencia documental, contrastar el contenido con las plantillas institucionales, organizar apartados y mejorar la redacción.

Las decisiones finales de alcance, arquitectura y diseño, así como la selección, adaptación, integración y validación del código incorporado al proyecto, fueron responsabilidad del autor. Todo resultado generado con apoyo de estas

herramientas fue revisado, ejecutado y verificado antes de su incorporación. El autor mantiene la responsabilidad sobre el funcionamiento del sistema y el contenido del documento, y no incorporó elementos que no pudiera explicar y defender técnicamente.

2.2.2 Evidencia de aplicación de la metodología

La aplicación de Kanban se evidencia en el tablero, el cronograma, los planes, la bitácora, el registro de errores y las pruebas. Estos elementos permiten reconstruir la planificación, ejecución y evidencia de cierre de cada tarjeta hasta el 7 de octubre de 2026.

2.3 Análisis de requisitos

Las Tablas 3 a 5 reúnen actores, requisitos, criterios y prioridad MoSCoW; la trazabilidad hacia las pruebas se amplía en el Anexo F.11.

2.3.1 Actores y perfiles de usuario

El sistema reconoce dos actores. Recepción administra pedidos y bebidas; Cocina prepara pedidos y administra pizzas y extras. La API mantiene esta separación aunque la interfaz oculte las acciones no autorizadas.

ID	Actor	Descripción	Qué puede hacer	Qué no puede hacer
A-01	Recepción	Personal que toma y gestiona pedidos desde el mostrador.	Iniciar y cerrar sesión; consultar la carta; registrar, seguir, cancelar cuando está pendiente, agregar según el estado, entregar pedidos y marcar bebidas como agotadas según RF-13.	Iniciar preparación o marcar listo; borrar pedidos físicamente; administrar usuarios, inventario, caja o sucursales.
A-02	Cocina	Personal que recibe y prepara pedidos en orden de llegada.	Iniciar y cerrar sesión; consultar la cola; pasar pedidos a en preparación y listo; y marcar pizzas o extras como agotados según RF-13.	Registrar ventas; entregar o cancelar pedidos; modificar precios; administrar usuarios o caja; borrar pedidos.

Tabla 3. Actores del sistema. Fuente: elaboración propia con base en los requisitos vigentes.

2.3.2 Requisitos funcionales

Cada requisito funcional incluye un camino esperado y otro de error o excepción. La prioridad MoSCoW comprende 7 Must, 5 Should y 2 Could.

ID	Historia de usuario	Criterios de aceptación	MoSCoW
RF-01	Como usuario, quiero iniciar sesión para acceder a mi rol.	CA-01.1 Credenciales válidas generan autenticación; la app valida el token y dirige al rol. CA-01.2 Credenciales inválidas no generan token ni muestran vistas de rol.	Must
RF-02	Como Recepción, quiero registrar un pedido para enviarlo a Cocina.	CA-02.1 Una selección válida crea el pedido pendiente y lo envía a Cocina. CA-02.2 Sin productos no se crea y se solicita agregar uno.	Must
RF-03	Como Recepción, quiero ver el estado en tiempo real sin consultar a Cocina.	CA-03.1 El cambio aparece en menos de 2 s sin recargar. CA-03.2 Si cae el canal, se avisa y puede recargarse el estado vigente.	Must
RF-04	Como Recepción, quiero recibir un aviso cuando un pedido esté listo.	CA-04.1 Al marcarse listo, aparece un aviso y se destaca el pedido. CA-04.2 Sin pedidos listos, se muestra el estado vacío.	Must
RF-05	Como Recepción, quiero marcar un pedido listo como entregado.	CA-05.1 Un pedido listo pasa a entregado y sale de la lista activa. CA-05.2 Si no está listo, se rechaza y se informa el estado.	Must
RF-06	Como Cocina, quiero recibir pedidos en tiempo real y por orden de llegada.	CA-06.1 El pedido aparece en menos de 2 s al final de la cola. CA-06.2 Sin pendientes, se muestra el estado vacío.	Must
RF-07	Como Cocina, quiero pasar un pedido a preparación y luego a listo.	CA-07.1 Cada transición válida se guarda y propaga. CA-07.2 Un pedido entregado o cancelado rechaza nuevos cambios.	Must
RF-08	Como usuario, quiero cerrar sesión y permitir otra en el dispositivo.	CA-08.1 Al cerrar, se descarta el token y vuelve el acceso. CA-08.2 Un token vencido rechaza la operación y exige autenticarse.	Should
RF-09	Como Recepción, quiero cancelar un pedido mientras esté pendiente.	CA-09.1 En pendiente y con motivo, se cancela y notifica a Cocina. CA-09.2 En preparación, listo o entregado, se impide cancelar.	Should
RF-10	Como Cocina, quiero ver la espera para priorizar pedidos antiguos.	CA-10.1 La tarjeta actualiza los minutos. CA-10.2 Al superar 15 minutos, el pedido activo muestra «demorado».	Could
RF-11	Como Recepción, quiero agregar una observación para Cocina.	CA-11.1 La nota aparece con el pedido. CA-11.2 Si supera el máximo, se rechaza e informa el límite.	Should

ID	Historia de usuario	Criterios de aceptación	MoSCoW
RF-12	Como Recepción, quiero visualizar un comprobante para el cliente.	CA-12.1 Presenta productos, cantidades, total y número. CA-12.2 Si no existe, informa «Pedido no encontrado».	Could
RF-13	Como usuario, quiero marcar productos agotados para no ofrecerlos.	CA-13.1 El rol autorizado cambia producto o categoría y todas las pantallas reflejan el estado. CA-13.2 Un rol sin permiso recibe 403 y no modifica datos.	Should
RF-14	Como Recepción, quiero agregar productos sin crear otro pedido.	CA-14.1 En pendiente o preparación admite pizzas y bebidas y avisa a Cocina. CA-14.2 En listo admite solo bebidas; estados finales no admiten agregados. Las líneas enviadas no se editan ni eliminan: se añaden nuevas.	Should

Tabla 4. Requisitos funcionales, criterios de aceptación y prioridad. Fuente: elaboración propia con base en el backlog vigente.

2.3.3 Requisitos no funcionales

Los cinco requisitos no funcionales definen criterios medibles; su ejecución y resultados se presentan en seguridad y pruebas.

ID	Tipo	Enunciado	Criterio de verificación
RNF-01	Rendimiento	Cambios sin intervención y listado fluido bajo la carga definida.	Estado < 2 s en 30 repeticiones, reportando el peor caso; GET /api/v1/pedidos < 2 s con 50 pedidos y 5 usuarios, medido con k6 y P95. Son criterios, no resultados.
RNF-02	Seguridad	Identidad con OpenID Connect y Keycloak, HTTPS público y autorización del servidor por rol.	Token válido; firma y expiración; rol y transición; transporte cifrado. Los resultados están en 2.7 y 2.8.
RNF-03	Usabilidad	Tarea principal en tres pasos o menos y vistas de datos con estado explícito.	Registro en hasta 3 pasos; estados cargando, con datos, vacío y error.
RNF-04	Compatibilidad	Recepción web adaptable en Chrome y Edge; Cocina como aplicación Android desde Flutter.	Sin desbordamiento a 1366 x 768 y 768 x 1024 en Chrome y Edge; APK 0.4.0 en Honor 50 Android 13 y Huawei Y9s Android 10.

ID	Tipo	Enunciado	Criterio de verificación
RNF-05	Disponibilidad	Servicio de 18:00 a 23:30 y degradación controlada al perder el canal.	GET /api/v1/salud cada 5 min por 7 días o más, con al menos 99 % de respuestas 200; aviso del canal < 10 s y recuperación o recarga. Son criterios, no resultados.

Tabla 5. Requisitos no funcionales y criterios de verificación. Fuente: elaboración propia.

2.3.4 Reglas de negocio

Las reglas de registro, precio, mitades, cancelación, agregados y disponibilidad se amplían en el Anexo F.1. Quedan fuera multisede, inventario, caja, reportes, promociones, administración y reparto.

2.3.5 Casos de uso

La Figura 2 relaciona los tres casos oficiales con ambos perfiles y representa la cancelación como alternativa del cambio de estado.

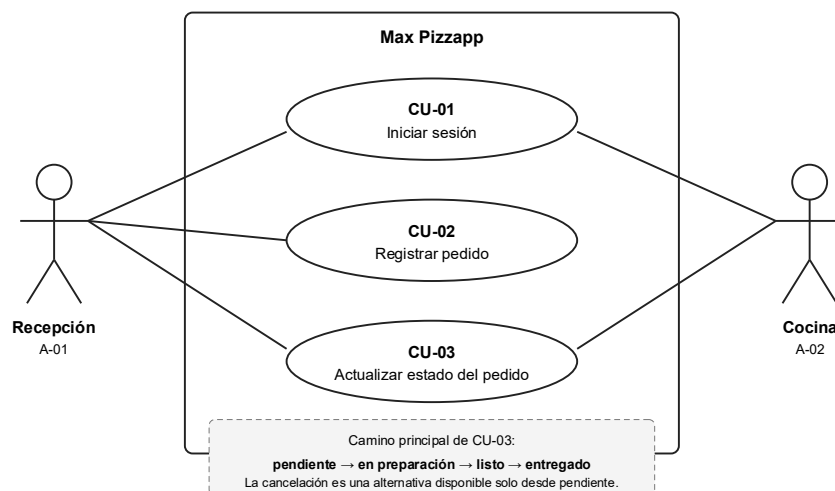


Figura 2. Diagrama de casos de uso de Max Pizzapp. Fuente: elaboración propia con base en los requisitos vigentes.

CU-01 Iniciar sesión

Actor principal: Recepción o Cocina. Precondición: cuenta activa en Keycloak y sistema accesible. Flujo: el cliente inicia la autorización; Keycloak valida las credenciales, devuelve un código y el cliente lo canjea con PKCE; la aplicación y la API validan el token y muestran la vista del rol. Alternativas: credenciales inválidas no generan token; un token vencido exige renovación o nuevo acceso. Postcondición: sesión autenticada por rol, sin que la contraseña pase por Max Pizzapp.

CU-02 Registrar pedido

Actor principal: Recepción. Precondición: sesión iniciada y carta disponible. Flujo: registra cliente y modalidad; agrega al menos una pizza, extras y, si corresponde, bebidas y observación; revisa el total y confirma. El servidor valida carta, reglas y precios, guarda el pedido pendiente y avisa a Cocina. Alternativas: datos inválidos o falta de pizza impiden crear; un cambio de precio o disponibilidad rechaza la confirmación. Postcondición: pedido, líneas, total, autor y número diario quedan persistidos y visibles para ambos perfiles.

CU-03 Actualizar el estado del pedido

Actor principal: Cocina; secundario: Recepción. Precondición: pedido activo y sesiones iniciadas. Flujo: Cocina recibe la cola, pasa el pedido de pendiente a en preparación y luego a listo; Recepción recibe el aviso y lo entrega. Cada cambio se valida, registra y propaga. Alternativas: Recepción cancela con motivo solo desde pendiente; una transición inválida se rechaza; tras una caída del canal, recarga o reconexión recupera el estado persistido. Postcondición: nuevo estado e historial con actor, nombre y fecha; cancelado y entregado son finales.

2.4 Diseño de la solución

2.4.1 Arquitectura del sistema

La arquitectura separa clientes, dominio, persistencia e identidad. Recepción usa principalmente la web y Cocina el destino Android; ambos proceden de una base Flutter común, aunque la web conserva las vistas compartidas.

Caddy cifra y encamina el acceso hacia API, Socket.IO y Keycloak. Node.js y Express aplican reglas; Socket.IO propaga eventos; Keycloak sostiene la identidad y PostgreSQL el estado recuperable. Ningún cliente accede directamente a la base; la arquitectura resultante se representa en la Figura 3.

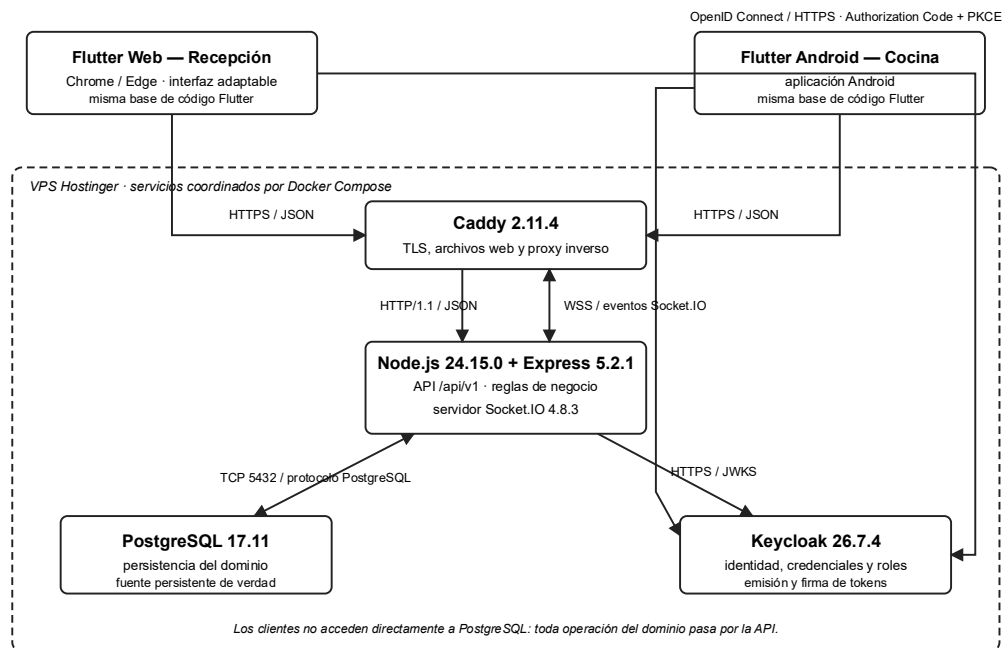


Figura 3. Arquitectura actual de Max Pizzapp. Fuente: elaboración propia con base en la configuración vigente del proyecto.

2.4.2 Modelo de datos

El modelo contiene cinco entidades. Keycloak administra usuarios fuera del dominio; pedido, detalle_pedido e historial_estado conservan el identificador y

nombre del actor. Las relaciones representan la segunda mitad y los extras de cada pizza. La Figura 4 representa este modelo vigente.

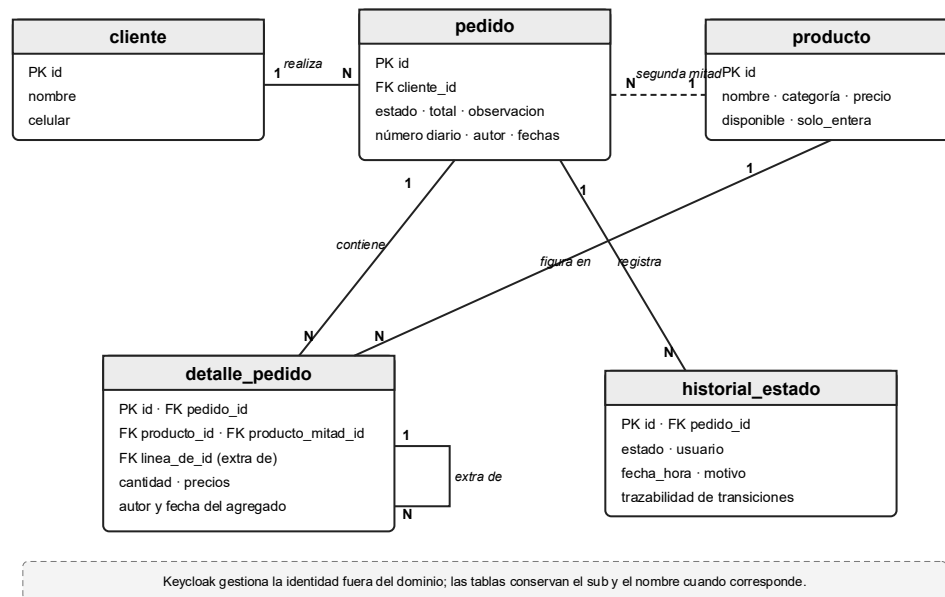


Figura 4. Modelo entidad-relación vigente. Fuente: elaboración propia a partir del esquema y sus migraciones.

PostgreSQL es la fuente persistente de verdad. Cancelar cambia el estado y conserva pedido e historial; no existe DELETE como operación normal. Las reglas de integridad del esquema no habilitan borrado en la interfaz ni en la API. Los diccionarios completos están en los Anexos F.3 a F.7.

2.4.3 Diseño de la interfaz

Recepción utiliza en computadora un formulario jerarquizado con cliente, modalidad, productos, observación, resumen y confirmación. Cocina prioriza lectura rápida en celular o tableta mediante una cola, tarjetas legibles, estado textual y una acción principal por etapa.

Las vistas distinguen cargando, con datos, vacío y error; los estados no dependen solo del color. El diseño contempla la etiqueta de demora de RF-10. La Figura 5 es un prototipo de baja fidelidad derivado de la interfaz vigente.

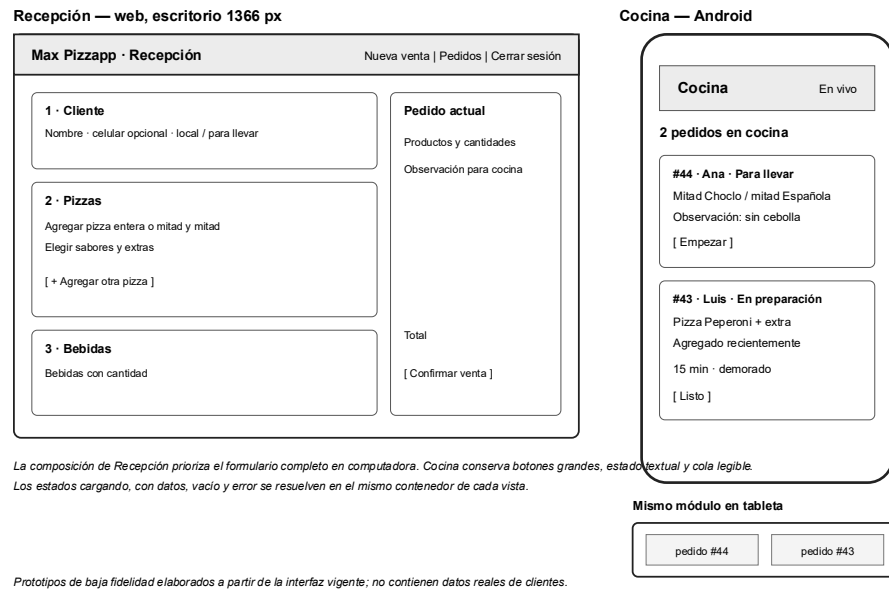


Figura 5. Prototipos de las pantallas principales y su adaptación. Fuente: elaboración propia a partir de la interfaz vigente.

2.4.4 Diseño de la interfaz de programación de aplicaciones

El contrato OpenAPI 3.0.3 define once operaciones bajo /api/v1, sintetizadas en la Tabla 6. Los clientes obtienen el token en Keycloak, no mediante /auth/login. RF-13 añadió operaciones para cambiar la disponibilidad por producto o categoría.

Método	Ruta	Auth	Rol	Entrada	2xx	Errores principales
GET	/api/v1/salud	No	Público	—	200 estado y base	503 degradado
GET	/api/v1/sesion	Bearer JWT	Recepción / Cocina	—	200 identidad y roles	401, 403
GET	/api/v1/productos	Bearer JWT	Recepción / Cocina	—	200 carta	400, 401, 403, 503

Método	Ruta	Auth	Rol	Entrada	2xx	Errores principales
GET	/api/v1/pedidos	Bearer JWT	Recepción / Cocina	estado opcional	200 lista activa	400, 401, 403, 503
POST	/api/v1/pedidos	Bearer JWT	Recepción	pedido o venta directa	201 pedido	400, 401, 403, 409, 503
GET	/api/v1/pedidos/{id}	Bearer JWT	Recepción / Cocina	id	200 pedido e historial	400, 401, 403, 404, 503
POST	/api/v1/pedidos/{id}/lineas	Bearer JWT	Recepción	líneas y total esperado	200 pedido actualizado	400, 401, 403, 404, 409, 503
PATCH	/api/v1/pedidos/{id}/estado	Bearer JWT	Según transición	estado y versión opcional	200 pedido actualizado	400, 401, 403, 404, 409, 503
POST	/api/v1/pedidos/{id}/cancelacion	Bearer JWT	Recepción	motivo	200 pedido cancelado	400, 401, 403, 404, 409, 503
PATCH	/api/v1/productos/{id}/disponibilidad	Sí	Según categoría	disponible	200 producto	400, 403, 404
PATCH	/api/v1/productos/disponibilidad	Sí	Según categoría	categoría, disponible	200 cantidad	400, 403

Tabla 6. Contrato vigente de la API. Fuente: elaboración propia a partir de docs/api/openapi.yaml.

No se expone DELETE para pedidos. La cancelación usa una operación específica, conserva el registro y solo procede desde pendiente; las transiciones y agregados también aplican reglas del dominio en el servidor.

2.5 Justificación del stack tecnológico

La Tabla 7 vincula las versiones verificadas con su función y con los requisitos del proyecto. La combinación separa interfaz, reglas, persistencia, identidad, eventos, entrada cifrada, contenedores y contrato API; su eficacia se sustenta en la implementación y las pruebas posteriores, no en la popularidad de cada tecnología.

Componente	Versión verificada	Función	Criterio de selección vinculado al proyecto
Flutter / Dart	3.44.8 / 3.12.2	Clientes web y Android.	Atiende RNF-04 y permite composiciones distintas sin duplicar el dominio de interfaz.
Node.js / Express	24.15.0 LTS / 5.2.1	API y reglas.	Centraliza validaciones de RF-02, RF-05, RF-07, RF-09 y RF-14 antes de persistir.
PostgreSQL	17.11	Persistencia relacional.	Sostiene cinco entidades, restricciones, transacciones e historial del objetivo 2.
Keycloak	26.7.4	Identidad y roles.	Separa la contraseña y aporta OIDC, tokens firmados y roles para RNF-02.
Socket.IO / socket_io_client	4.8.3 / 3.1.6	Eventos y reconexión.	Atiende RF-03, RF-04 y RF-06; la persistencia recupera el estado tras una caída.
flutter_appauth	12.1.0	OIDC en Android.	Usa navegador del sistema y Authorization Code + PKCE sin recoger la contraseña.
Caddy	2.11.4	HTTPS, web y proxy.	Expone una entrada cifrada y rutas para API, Socket.IO y Keycloak, según RNF-02.
Docker Engine / Compose	29.7.2 dev; 29.8.1 servidor / v5.3.1 dev; v5.5.1 servidor	Coordina servicios.	Declara API, base, identidad y proxy con redes y volúmenes; reproduce RNF-05.
OpenAPI	3.0.3; contrato 1.0.0	Contrato de API.	Vincula diseño y rutas reales y permite contrastar cliente, servidor y objetivo 2.

Tabla 7. Stack tecnológico y criterios de selección. Fuente: elaboración propia con base en la configuración vigente del proyecto.

2.6 Implementación

Este apartado resume lo construido y verificado hasta el 7 de octubre de 2026, organizado por responsabilidades y tarjetas Kanban.

2.6.1 Organización del código

El repositorio separa backend, frontend, contenedores, pruebas, scripts y documentación. La API concentra el dominio, Flutter comparte lógica web y Android, y Docker reúne esquema, migraciones, identidad y proxy. Los secretos se suministran por variables de entorno.

2.6.2 Avance por tarjeta

El Anexo F.8 relaciona tarjetas y requisitos. Al 8 de octubre, las tarjetas 08 y 14 incorporaron disponibilidad y bebidas reales; la 11 actualizó el acceso y la 13 cerró la instalación local con una prueba externa.

2.6.3 El flujo implementado: del mostrador a la cocina

Recepción registra cliente, modalidad, pizzas, bebidas y observación. La API verifica rol, recalcula precios y guarda pedido, detalles e historial en una transacción; luego emite pedido:nuevo. Cocina pasa el pedido a en preparación y listo, y Recepción lo entrega. Los agregados admiten pizzas y bebidas en pendiente o preparación y solo bebidas en listo; los estados finales no admiten cambios. La cancelación es lógica y exclusiva de pendiente. RF-13 propaga la disponibilidad por eventos y autoriza por categoría. Ante una caída del canal, la interfaz avisa y Recargar consulta la API.

2.6.4 Decisiones de implementación

El servidor calcula importes en centavos, rechaza totales divergentes y confirma cada transacción antes de emitir eventos. Estado, rol y versión del pedido protegen las transiciones y agregados; el número diario se asigna bajo bloqueo transaccional con la hora de Bolivia.

La configuración establece 4 s de latido y 3 s de espera: un máximo teórico de 7 s para detectar una pérdida silenciosa y aproximadamente 1 s antes de mostrar la banda. El peor valor medido fue 6,89 s. El canal termina al vencer el token y el APK usa AppAuth con PKCE.

2.6.5 Interfaz implementada

El formulario de venta reúne cliente, modalidad, pizzas, bebidas, observación, resumen y confirmación. La Figura 6 incorpora la carta vigente: Agua 500 ml, Bs 6; Jugo 1 L, Bs 15; y sodas personal, 1 L, 1,5 L y 2 L a Bs 3, 10, 15 y 20. Los precios de las quince pizzas están confirmados; los extras siguen provisionales.

Figura 6. Formulario de venta de Recepción con pizzas y bebidas reales. Fuente: captura del sistema desplegado, 07/10/2026.

Cocina utiliza el APK 0.4.0 de la base Flutter, validado físicamente en un Honor 50 con Android 13; la interfaz móvil se muestra en la Figura 7. La cola ordena y avisa pedidos, permite cambiar estados y, desde Carta, administrar pizzas y extras; Recepción administra bebidas.



Figura 7. APK 0.4.0 de Cocina con un pedido que incluye una bebida. Fuente: captura del sistema desplegado, 07/10/2026.

En Pedidos, Recepción cancela solo desde pendiente; agrega pizzas y bebidas en pendiente o preparación, y solo bebidas en listo; las líneas enviadas no se editan ni eliminan, sino que se añaden nuevas. Desde listo puede entregar o llamar al cliente. Las vistas implementan estados de carga, datos, vacío y error con reintento, y las pruebas verifican el contraste según WCAG.

2.6.6 Fragmentos de código ilustrativos

Las transiciones y el rol autorizado se concentran en una tabla del servidor, reproducida en la Figura 8:

```
const TRANSICIONES = {  
  en_preparacion: { desde: ['pendiente'], rol: 'cocina' },  
  listo: { desde: ['en_preparacion'], rol: 'cocina' },  
  entregado: { desde: ['listo'], rol: 'recepcion' },  
  cancelado: { desde: ['pendiente'], rol: 'recepcion' },  
};
```

Figura 8. Las transiciones de estado y el rol que hace cada una. Fuente:
backend/src/pedidos/estados.js.

El Anexo F.10 conserva el latido y el corte por vencimiento del token. En el cuerpo queda la tabla de transiciones por resumir la autorización central del flujo.

2.6.7 Dificultades y su resolución

Las pruebas revelaron conflictos entre Express y Socket.IO, un evento que no cruzaba la API, demora del latido, respuestas 500 ante entradas inválidas y un campo de Keycloak sobredimensionado. Cada defecto se reprodujo, corrigió y cubrió con una prueba. RF-13 añadió autorización por categoría: Cocina administra pizzas y extras, Recepción bebidas, y la API responde 403 ROL_SIN_PERMISO ante cruces; sus eventos mantienen alineadas las pantallas.

2.7 Seguridad

La seguridad combina identidad centralizada, autorización del servidor, validación de entradas y cifrado, verificados mediante suites, sondas, configuraciones y casos CP-15 a CP-22, CP-25 y CP-31.

2.7.1 Autenticación y sesiones

Keycloak 26.7.4 usa OpenID Connect, código de autorización y PKCE; la contraseña no pasa por Max Pizzapp. La API responde 401 ante tokens

ausentes, alterados, vencidos o de otro contexto, y el canal en vivo no supera su vigencia.

2.7.2 Autorización por rol

Cada ruta comprueba el rol antes de acceder a la base. Cocina controla preparación, pizzas y extras; Recepción registra, cancela, agrega, entrega y controla bebidas. Una cuenta sin rol recibe 403; ocultar botones no sustituye la autorización del servidor.

2.7.3 Entradas y exposición

La aplicación valida para orientar al usuario, la API vuelve a validar como frontera confiable y PostgreSQL aplica restricciones, como resume la Tabla 8. Se usan consultas parametrizadas, límites de cuerpo y peticiones, errores sin trazas, Helmet y CSP. La matriz OWASP está en el Anexo F.9.

2.7.4 Cifrado y secretos

Caddy termina TLS y fuerza HTTPS con HSTS, comprobados el 6 de octubre de 2026. Keycloak conserva contraseñas con Argon2id. Las credenciales residen en el entorno restringido del servidor; la documentación pública solo identifica variables.

Dato	En la app	En la API y en la base	Si no cumple
Nombre del cliente	Obligatorio; hasta 120 caracteres	Obligatorio, hasta 120, sin caracteres de control; restricción de no vacío	400 VENTA_INVALIDA
Celular	Opcional; 8 dígitos que empiezan con 5, 6 o 7	Lo mismo; restricción <code>^[5-7][0-9]{7}\$</code> y único	400 VENTA_INVALIDA
Líneas, mitades y extras	Se eligen de la carta; al menos una pizza	Que existan y estén disponibles; una pizza <i>solo entera</i> no va por mitades; claves foráneas	400; 409 PRODUCTO_NO_DISPONIBLE
Cantidad	De 1 a 999, con botones	Entero de 1 a 999; restricción en la base	400 VENTA_INVALIDA
Total	Calculado con la misma regla	Recalculado por el servidor; si no coincide, no se guarda nada	409 PRECIO_CAMBIADO

Dato	En la app	En la API y en la base	Si no cumple
Estado y motivo	Solo el botón que corresponde al rol; motivo obligatorio para cancelar	Rol, transición, versión del pedido y motivo de hasta 120 caracteres	400; 403; 409
Cuerpo de la petición	Lo arma la app	JSON en UTF-8 de hasta 100 KB	400; 413; 415

Tabla 8. Validación doble de los datos principales. Fuente: implementación y casos de prueba al 07/10/2026.

2.8 Pruebas

Los reportes identifican fecha, versión y entorno, de modo que los resultados pueden contrastarse con la evidencia técnica.

2.8.1 Plan de pruebas

Las pruebas avanzaron desde unidades aisladas hasta producción. La Tabla 9 resume niveles y reportes tras incorporar RF-13 y las bebidas reales.

Nivel	Qué prueba	Herramienta	Alcance	Reporte
API	Rutas, token y rol, validación, precio, estados, canal en vivo y contrato OpenAPI, con la base simulada	node:test (Node 24.15)	332 pruebas aprobadas	api.txt, api-junit.xml
App	Venta, pedidos, cocina, acceso en la web y en el APK, sesión, canal y sus cortes, avisos, anchos y contraste, con la API simulada	flutter_test (Flutter 3.44.8)	338 pruebas aprobadas y análisis sin advertencias	app.txt
Punta a punta	Acceso por PKCE, salud, carta, pedidos, errores y disponibilidad contra el entorno real	Python 3	6 sondas	sonda-*.txt
Tiempo real	Del cambio a la llegada del aviso en la otra pantalla	socket.io-client	30 repeticiones	tiempo-real.txt
Carga	El listado con 50 pedidos activos y 5 usuarios a la vez	k6 2.3.0, en Docker	60 s	carga-k6.txt
Disponibilidad	GET /api/v1/salud, las 24 horas	UptimeRobot	Cada 5 min desde el 23-sep	Panel del monitor

Nivel	Qué prueba	Herramienta	Alcance	Reporte
Integración continua	Las dos suites, en un clon limpio	GitHub Actions	Cada push	Corridas en el repositorio

Tabla 9. Niveles de prueba al 8 de octubre de 2026. Fuente: elaboración propia a partir de docs/pruebas/README.md.

API y app se prueban sin base, Keycloak ni .env: se simulan consultas y se firman tokens localmente. Las sondas, el tiempo real y la carga usan cuentas de prueba y eliminan los pedidos ficticios al terminar.

2.8.2 Pruebas automatizadas

La API aprobó 332/332 pruebas; la app, 338/338, y flutter analyze no produjo advertencias. Seis sondas cubrieron acceso, salud, carta, pedido, errores y disponibilidad; la última confirmó 25/25 respuestas.

2.8.3 Casos de prueba

La matriz CP-01 a CP-31 está en el Anexo F.11; la Tabla 10 resume su alcance y resultado.

Grupo	Cobertura	Resultado
CP-01–CP-14	Criterios de los siete requisitos Must	14 aprobados
CP-15–CP-22	Autenticación, autorización, entradas y controles	8 aprobados
CP-23–CP-28	Requisitos no funcionales; CP-25 aprobado	6 aprobados
CP-29–CP-31	Disponibilidad de productos y autorización por categoría	3 aprobados

Tabla 10. Síntesis de casos de prueba al 8 de octubre de 2026. Fuente: elaboración propia a partir de los reportes versionados.

2.8.4 Requisitos no funcionales

RNF-01. En producción se midieron 30 repeticiones por evento, desde la confirmación hasta el aviso por Socket.IO (Tabla 11).

Evento	Mínimo	Mediana	P95	Máximo	Límite
Pedido nuevo -> Cocina	154 ms	156 ms	159 ms	162 ms	< 2 s
Estado -> Recepción	153 ms	155 ms	159 ms	450 ms	< 2 s
Agregado -> Cocina	153 ms	157 ms	164 ms	166 ms	< 2 s
Disponibilidad -> Recepción	156 ms	158 ms	163 ms	315 ms	< 2 s

Tabla 11. Propagación de eventos en producción, 07/10/2026. Fuente: docs/pruebas/reportes/tiempo-real.txt.

El máximo observado fue 450 ms, menor al límite de 2 s. Con 50 pedidos y 5 usuarios, k6 registró 235 lecturas, P95 de 296,36 ms, máximo de 369,42 ms y ninguna falla.

RNF-03. La venta comprende cliente, productos y confirmación (D-60); exige cliente y al menos una pizza, mientras bebidas y observación son opcionales. Una prueba completó la venta en tres pasos sin cambiar de pantalla. Los estados cargando, con datos, vacío y error se aprobaron en las tres vistas: 12 de 12.

RNF-04. Las vistas no desbordaron a 1366 x 768 ni 768 x 1024. La verificación física cubrió Edge 154 y Chrome 154.0.8037.98 en ambos tamaños, y el APK 0.4.0 en Honor 50 con Android 13 y Huawei Y9s con Android 10; CP-27 quedó aprobado. La Figura 9 muestra las tres vistas en la resolución de tableta.

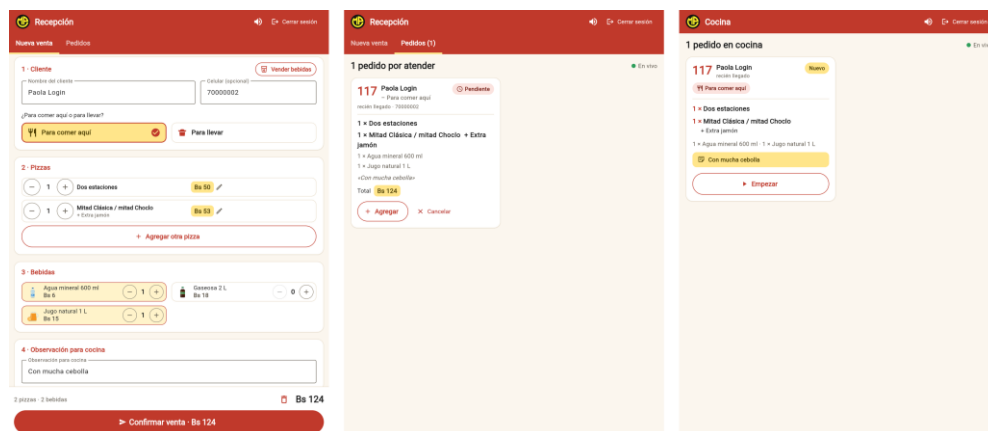


Figura 9. La venta, los pedidos de Recepción y la cola de Cocina a 768 × 1024 px, en Edge 154. Fuente: captura del sistema desplegado, 02/10/2026.

RNF-05. UptimeRobot consulta `/api/v1/salud` cada cinco minutos. El caso CP-28 se cerró con 100 % durante 9 días y 20 horas, entre el 23 de septiembre de 2026 a la 01:19 y el 2 de octubre de 2026 a las 21:48, sin incidentes ni minutos de caída registrados. Una comprobación posterior realizada el 10 de octubre mostró que el mismo monitor continuaba operativo, con aproximadamente 17 días y 11 horas de continuidad al 100 % y sin incidentes registrados. El sondeo puede omitir cortes menores al intervalo de cinco minutos, como los tres minutos del mantenimiento del 8 de octubre. La pérdida silenciosa tiene un máximo teórico de 7 s más cerca de 1 s visual; el peor valor medido fue 6,89 s más esa espera.

2.8.5 Alcance y límites de las pruebas

No se automatizó navegador porque Flutter dibuja en lienzo y vuelve frágil Playwright o Selenium; el recorrido se cubrió con sondas, pruebas de la app y comprobaciones web y APK. Tampoco se midió cobertura: la matriz vincula pruebas con requisitos. RNF-01 evaluó la carga prevista para un local, no estrés.

2.9 Despliegue

Max Pizzapp está disponible en <https://maxpizzapp.tech> desde el 23 de septiembre de 2026, en un VPS Hostinger KVM 1 con Ubuntu 24.04 LTS, 1 vCPU, 4 GB de memoria y 50 GB de disco. Un solo servidor simplifica el piloto, sin aportar redundancia.

2.9.1 Plataforma y servicios

Docker Compose declara Caddy, API, Keycloak y PostgreSQL. Caddy publica 80/443 y encamina web, API, Socket.IO e identidad; base y Keycloak quedan en la red interna. El acceso administrativo usa llave SSH y las versiones están fijadas.

2.9.2 Procedimiento y verificación

Para el despliegue se configuraron el dominio y el entorno, se validaron Compose y Caddy, se levantaron los servicios, se endureció Keycloak y se realizaron comprobaciones desde otra red. Las actualizaciones son manuales y fuera del horario; el 8 de octubre, las seis sondas aprobaron tras el mantenimiento. La integración continua prueba un clon limpio, pero no publica: una migración requiere respaldo y ventana controlada. El procedimiento completo está en el Anexo B y las comprobaciones externas se sintetizan en la Tabla 12.

Comprobación	Resultado
https://maxpizzapp.tech/	200 por HTTPS; HTTP redirige de forma permanente.
/api/v1/salud	200; incluida en la sonda de salud.
Identidad	Emisor https://auth.maxpizzapp.tech/realms/maxpizzapp .
Cuentas de prueba	Autenticación por PKCE y recorrido de venta a entrega.
Sondas	Seis recorridos: acceso, salud, carta, pedidos, errores y disponibilidad.

Tabla 12. Verificación externa del despliegue. Fuente: comprobaciones del autor y reportes de producción, 05–08/10/2026.

2.9.3 Entornos y distribución

Entorno	Propósito	Ejecución	Dirección
Desarrollo	Programación y pruebas	Compose de desarrollo y Flutter	localhost
Instalación local	Instancia completa en una PC	Instalador y perfil completo	http://localhost:8090
Producción	Servicio público del piloto	Compose de producción y HTTPS	https://maxpizzapp.tech

Tabla 13. Entornos de ejecución. Fuente: configuración vigente del proyecto.

La Tabla 13 diferencia los entornos de desarrollo, instalación local y producción. El APK universal 0.4.0 se distribuye desde la versión pública, con firma externa al repositorio y huella SHA-256. La instalación local publica solo en 127.0.0.1. El 8 de octubre, el encargado de recepción instaló en Windows

11 y completó una venta hasta la entrega; las intervenciones sobre WSL, ventana privada y contraseña quedaron incorporadas al Anexo B.

2.9.4 Monitoreo y limitaciones

UptimeRobot consulta la ruta de salud de la API cada cinco minutos. El caso CP-28 se cerró con un intervalo formal de 9 días y 20 horas al 100 %, entre el 23 de septiembre y el 2 de octubre, sin incidentes ni minutos de caída registrados. Como evidencia posterior, la captura detallada del 10 de octubre muestra que el mismo monitor continuaba activo con aproximadamente 17 días y 11 horas de continuidad al 100 % y sin incidentes registrados; la Figura F.5 muestra ambos servicios como operativos en la página de estado pública. La vista denominada *Last 30 days* no representa treinta días efectivamente medidos, porque el monitor tenía una antigüedad menor. El sondeo cada cinco minutos puede no detectar interrupciones más breves que ese intervalo. El incidente IPv4 del 8 de octubre mostró que una sonda externa puede omitir problemas de un proveedor. Persisten servidor único, respaldos manuales y restauración no ensayada.

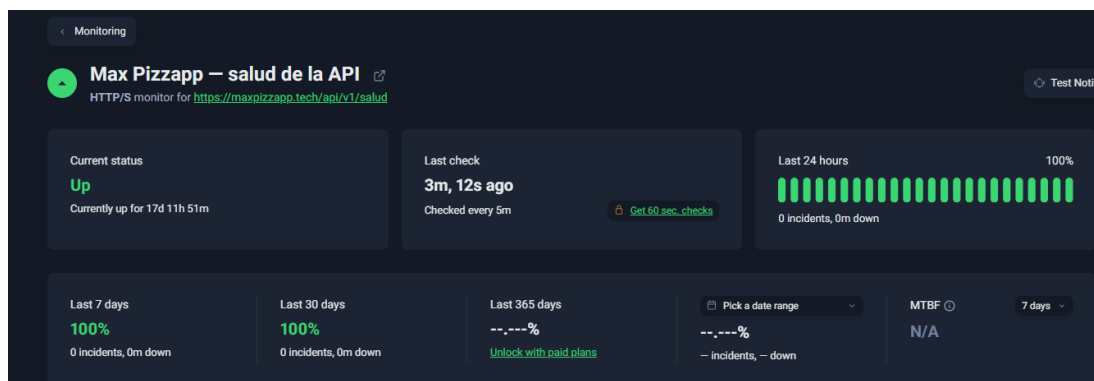


Figura 10. Monitor de salud de la API al 10 de octubre de 2026. Fuente: UptimeRobot.

Estas limitaciones delimitan el alcance de la validación realizada y orientan las recomendaciones formuladas en el capítulo siguiente.

CAPÍTULO 3

CAPÍTULO 3. CONCLUSIONES Y RECOMENDACIONES

Este capítulo contrasta los resultados con los seis objetivos, usando la evidencia de 2.3 a 2.9 y sin atribuir efectos operativos no medidos. El grado de cumplimiento se sintetiza en la Tabla 14.

3.1 Conclusiones

Objetivo	Grado	Evidencia principal
1. Requisitos	Cumplido	14 RF y 5 RNF; criterios trazables a casos.
2. Diseño	Cumplido	Arquitectura, cinco entidades y contrato de 11 operaciones.
3. Implementación	Cumplido	7 Must y 5 Should; módulos sincronizados.
4. Seguridad	Cumplido	OIDC con PKCE; 401/403; controles probados.
5. Pruebas	Cumplido	31 casos aprobados; RNF-04 verificado en dos navegadores y dos teléfonos.
6. Despliegue	Cumplido	Servicio público, manuales y prueba externa del 08/10/2026 completada.

Tabla 14. Grado de cumplimiento de los objetivos específicos. Fuente: elaboración propia.

El objetivo 1 se cumplió: se definieron 14 requisitos funcionales y 5 no funcionales, con criterios verificables, prioridad MoSCoW y trazabilidad hacia las pruebas. RF-13 y RF-14 se incorporaron sin ampliar el piloto fuera del flujo Recepción-Cocina.

El objetivo 2 se cumplió: la arquitectura separa clientes, servidor, persistencia e identidad; el modelo contiene cinco entidades y OpenAPI documenta once operaciones. Los cambios se reflejaron en el modelo y contrato vigentes.

El objetivo 3 se cumplió: Recepción y Cocina comparten el pedido y reciben cambios sin recargar. Se implementaron 7 Must y 5 Should; en 30 repeticiones por evento, el máximo fue 450 ms, inferior a 2 s.

El objetivo 4 se cumplió: Keycloak autentica con OpenID Connect y PKCE, la API verifica token y rol, y HTTPS cifra la comunicación. Las pruebas confirmaron 401 y 403, incluida RF-13. No se realizó una auditoría externa.

El objetivo 5 se cumplió: API y app aprobaron 332 y 338 pruebas; se ejecutaron seis sondas, 25 comprobaciones de error, tiempo real y carga, con 31 casos aprobados. CP-27 verificó dos tamaños en Edge y Chrome y el APK 0.4.0 en Honor 50 Android 13 y Huawei Y9s Android 10.

El objetivo 6 se cumplió: el sistema está disponible públicamente y cuenta con manuales. El 8 de octubre, el encargado de recepción instaló en Windows 11 y completó el flujo en 1 hora y 30 minutos, incluida la incidencia de WSL. Las tres intervenciones del autor condujeron a corregir el manual, por lo que la prueba demuestra el procedimiento y delimita su autonomía.

Max Pizzapp comparte el pedido, registra cambios y separa acciones de Recepción y Cocina. La evidencia demuestra funcionamiento y trazabilidad, no una reducción cuantitativa de demoras, errores o tiempos.

3.2 Recomendaciones

Repetir la instalación con el manual corregido en otra computadora y, si el piloto lo requiere, en Windows 10. Mantener actualizadas las indicaciones sobre WSL, ventana privada y contraseña cuando cambie Docker Desktop.

Medir durante dos a cuatro semanas de uso real los tiempos entre estados y la frecuencia de incidencias para contrastar las estimaciones iniciales.

Completar RF-10 con la señal de demora y evaluar RF-12 solo si el negocio requiere comprobante. Sustituir los precios provisionales de extras cuando existan datos reales.

Antes de la adopción sostenida, crear cuentas individuales, retirar las de demostración y capacitar a ambos perfiles. Programar respaldos, mantener alertas y conservar el papel como contingencia.

BIBLIOGRAFÍA CONSULTADA

BASS, Len; CLEMENTS, Paul y KAZMAN, Rick. Software Architecture in Practice. 4.^a ed. Boston: Addison-Wesley Professional, 2021. ISBN: 978-0-13-688609-9.

BIRYUKOV, Alex; DINU, Daniel; KHOVRATOVICH, Dmitry y JOSEFSSON, Simon. Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications. RFC 9106 [en línea]. IETF, 2021. DOI: 10.17487/RFC9106. Disponible en: <https://www.rfc-editor.org/info/rfc9106> [consulta: 2 de octubre de 2026].

CADDY. Automatic HTTPS [en línea]. [s. f.]. Disponible en: <https://caddyserver.com/docs/automatic-https> [consulta: 7 de octubre de 2026].

CODD, Edgar F. A relational model of data for large shared data banks. Communications of the ACM [en línea]. 1970, vol. 13, n.º 6, pp. 377-387. DOI: 10.1145/362384.362685. Disponible en: <https://doi.org/10.1145/362384.362685> [consulta: 29 de septiembre de 2026].

DENNISS, William y BRADLEY, John. OAuth 2.0 for Native Apps. RFC 8252 [en línea]. IETF, 2017. DOI: 10.17487/RFC8252. Disponible en: <https://www.rfc-editor.org/info/rfc8252> [consulta: 2 de octubre de 2026].

DOCKER. How Compose works [en línea]. [s. f.a]. Disponible en: <https://docs.docker.com/compose/intro/compose-application-model/> [consulta: 30 de septiembre de 2026].

DOCKER. What is a container? [en línea]. [s. f.b]. Disponible en: <https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/> [consulta: 30 de septiembre de 2026].

DUSSEAU, Lisa y SNELL, James. PATCH Method for HTTP. RFC 5789 [en línea]. IETF, 2010. DOI: 10.17487/RFC5789. Disponible en: <https://www.rfc-editor.org/info/rfc5789> [consulta: 29 de septiembre de 2026].

FETTE, Ian y MELNIKOV, Alexey. The WebSocket Protocol. RFC 6455 [en línea]. IETF, 2011. DOI: 10.17487/RFC6455. Disponible en: <https://www.rfc-editor.org/info/rfc6455> [consulta: 29 de septiembre de 2026].

FIELDING, Roy T.; NOTTINGHAM, Mark y RESCHKE, Julian. HTTP Semantics. RFC 9110 [en línea]. IETF, 2022. DOI: 10.17487/RFC9110. Disponible en: <https://www.rfc-editor.org/info/rfc9110> [consulta: 29 de septiembre de 2026].

FLUTTER. General approach to adaptive apps [en línea]. 2026a. Disponible en: <https://docs.flutter.dev/ui/adaptive-responsive/general> [consulta: 30 de septiembre de 2026].

FLUTTER. Supported deployment platforms [en línea]. 2026b. Disponible en: <https://docs.flutter.dev/reference/supported-platforms> [consulta: 30 de septiembre de 2026].

FLUTTER. Testing Flutter apps [en línea]. 2026c. Disponible en: <https://docs.flutter.dev/testing/overview> [consulta: 2 de octubre de 2026].

FUDO. Funcionalidades: ¿Qué puedo hacer con Fudo? [en línea]. [s. f.]. Disponible en: <https://fu.do/es-ar/funcionalidades/> [consulta: 28 de septiembre de 2026].

GITHUB. GitHub Actions documentation [en línea]. [s. f.]. Disponible en: <https://docs.github.com/en/actions> [consulta: 2 de octubre de 2026].

GRAFANA LABS. Thresholds. Grafana k6 documentation [en línea]. [s. f.]. Disponible en: <https://grafana.com/docs/k6/latest/using-k6/thresholds/> [consulta: 2 de octubre de 2026].

HODGES, Jeff; JACKSON, Collin y BARTH, Adam. HTTP Strict Transport Security (HSTS). RFC 6797 [en línea]. IETF, 2012. DOI: 10.17487/RFC6797. Disponible en: <https://www.rfc-editor.org/info/rfc6797> [consulta: 2 de octubre de 2026].

JONES, Michael; BRADLEY, John y SAKIMURA, Nat. JSON Web Token (JWT). RFC 7519 [en línea]. IETF, 2015. DOI: 10.17487/RFC7519. Disponible en: <https://www.rfc-editor.org/info/rfc7519> [consulta: 2 de octubre de 2026].

KEYCLOAK. Securing applications and services with OpenID Connect [en línea]. [s. f.]. Disponible en: <https://www.keycloak.org/securing-apps/oidc-layers> [consulta: 30 de septiembre de 2026].

LOYVERSE. Sistema de pantalla de cocina para POS: Loyverse KDS [en línea]. [s. f.]. Disponible en: <https://loyverse.com/es/kitchen-display-system> [consulta: 28 de septiembre de 2026].

NODE.JS. Test runner. Node.js v24 documentation [en línea]. [s. f.]. Disponible en: <https://nodejs.org/docs/latest-v24.x/api/test.html> [consulta: 2 de octubre de 2026].

OPEN SOURCE POINT OF SALE. Open Source Point of Sale [en línea]. [s. f.]. Disponible en: <https://github.com/opensourcepos/opensourcepos> [consulta: 28 de septiembre de 2026].

OPEN SOURCE POINT OF SALE. Releases [en línea]. [s. f.]. Disponible en: <https://github.com/opensourcepos/opensourcepos/releases> [consulta: 28 de septiembre de 2026].

OPENAPI INITIATIVE. OpenAPI Specification, version 3.0.3 [en línea]. 2020. Disponible en: <https://spec.openapis.org/oas/v3.0.3> [consulta: 29 de septiembre de 2026].

ORACLE. KDS Basics [en línea]. [s. f.]. Disponible en: https://docs.oracle.com/en/industries/food-beverage/symphony/19.8/kdscu/c_kds_basics.htm [consulta: 28 de septiembre de 2026].

OWASP FOUNDATION. OWASP Top 10:2025 [en línea]. 2025. Disponible en: <https://owasp.org/Top10/2025/> [consulta: 2 de octubre de 2026].

POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL 17.11 Documentation: 5.5 Constraints [en línea]. [s. f.b]. Disponible en: <https://www.postgresql.org/docs/17/ddl-constraints.html> [consulta: 30 de septiembre de 2026].

POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL 17.11 Documentation: BEGIN [en línea]. [s. f.a]. Disponible en:

<https://www.postgresql.org/docs/17/sql-begin.html> [consulta: 30 de septiembre de 2026].

SAKIMURA, Nat; BRADLEY, John y AGARWAL, Naveen. Proof Key for Code Exchange by OAuth Public Clients. RFC 7636 [en línea]. IETF, 2015. DOI: 10.17487/RFC7636. Disponible en: <https://www.rfc-editor.org/info/rfc7636> [consulta: 29 de septiembre de 2026].

SAKIMURA, Nat; BRADLEY, John; JONES, Michael; DE MEDEIROS, Breno y MORTIMORE, Chuck. OpenID Connect Core 1.0 incorporating errata set 2 [en línea]. OpenID Foundation, 2023. Disponible en: https://openid.net/specs/openid-connect-core-1_0.html [consulta: 29 de septiembre de 2026].

SOCKET.IO. Introduction [en línea]. 2026a. Disponible en: <https://socket.io/docs/v4/> [consulta: 30 de septiembre de 2026].

SOCKET.IO. Using multiple nodes [en línea]. 2026b. Disponible en: <https://socket.io/docs/v4/using-multiple-nodes/> [consulta: 7 de octubre de 2026].

TAN, Tan-Hsu y CHANG, Ching-Su. Development and evaluation of an RFID-based e-restaurant system for customer-centric service. Expert Systems with Applications [en línea]. 2010, vol. 37, n.º 9, pp. 6482-6492. DOI: 10.1016/j.eswa.2010.02.137. Disponible en: <https://doi.org/10.1016/j.eswa.2010.02.137> [consulta: 28 de septiembre de 2026].

W3C. Content Security Policy Level 3 [en línea]. [s. f.]. Disponible en: <https://www.w3.org/TR/CSP3/> [consulta: 2 de octubre de 2026].

ANEXOS

ANEXO A. MANUAL DE USUARIO

Max Pizzapp tiene dos perfiles: Recepción registra ventas, sigue pedidos y entrega; Cocina recibe la cola, inicia la preparación y marca los pedidos listos. Las instrucciones siguientes describen tareas de uso y emplean nombres y datos personales de demostración; los productos y precios corresponden al corte documentado del sistema, salvo los extras provisionales.

A.1 Acceso al sistema

1. En una computadora, abra <https://maxpizzapp.tech>. En el teléfono de Cocina, abra Max Pizzapp Cocina.
2. Toque Iniciar sesión. En la pantalla de acceso escriba el usuario de su perfil y la contraseña asignada.
3. Compruebe que la barra superior diga Recepción o Cocina. Para salir, use Cerrar sesión.

Qué debe observar: la página de acceso muestra la identidad de Max's Pizzas; la contraseña solo se introduce en esa página. La aplicación abre las funciones del perfil reconocido.

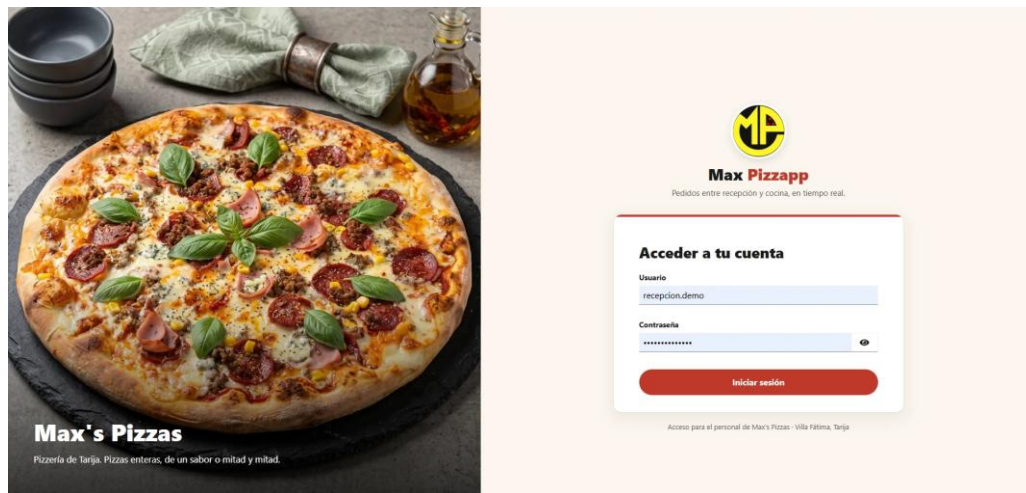


Figura A.1. Pantalla de acceso con la identidad visual vigente. Fuente: captura del sistema desplegado, 08/10/2026.

A.2 Recepción

A.2.1 Registrar una venta

1. En Nueva venta, escriba el nombre del cliente. El celular es opcional; si se registra, debe tener ocho dígitos y empezar con 5, 6 o 7.
2. Elija Para comer aquí o Para llevar.
3. Toque Agregar pizza. Elija Entera para un sabor o Mitad y mitad para dos sabores distintos. Puede añadir extras y ajustar la cantidad. Dos estaciones, Tres estaciones y Criolla española solo se ofrecen enteras.
4. Añada bebidas si corresponde. La marca de la soda y el sabor del jugo se preguntan al entregar y no se registran.
5. Escriba una observación para Cocina si hace falta, revise el resumen y toque Confirmar venta.

Qué debe observar: el resumen muestra productos, cantidades y total. Al confirmar, la pantalla queda lista para otra venta y el pedido aparece en Cocina.

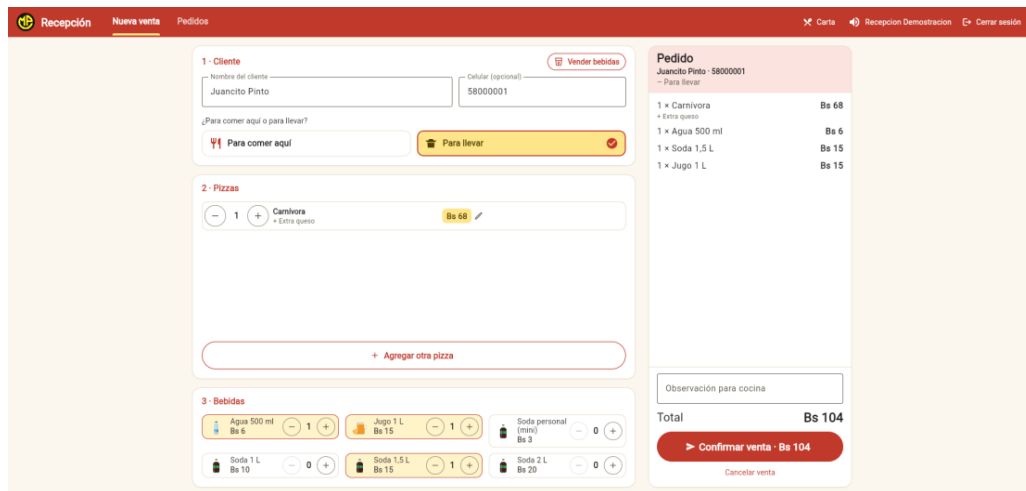


Figura A.2. Venta preparada con pizza, bebidas y total. Fuente: captura del sistema desplegado, 07/10/2026.

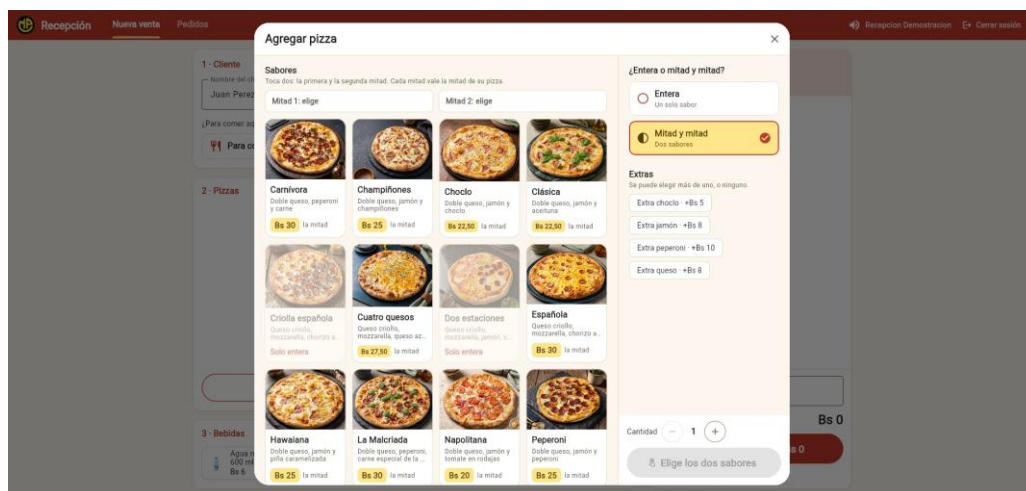


Figura A.3. Selección de una pizza de dos mitades. Fuente: captura del sistema desplegado, 25/09/2026.

A.2.2 Vender solo bebidas

1. En Nueva venta, toque Vender bebidas.
2. Sume las bebidas y toque Cobrar. Esta venta no lleva cliente ni pasa por Cocina; queda entregada en el mostrador.

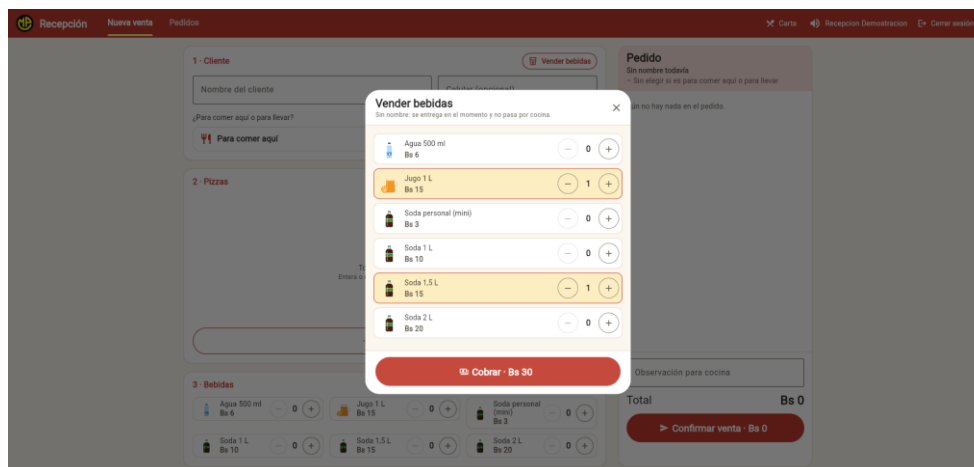


Figura A.4. Venta directa de bebidas. Fuente: captura del sistema desplegado, 07/10/2026.

A.2.3 Seguir, modificar y cerrar pedidos

1. Abra Pedidos. Cada tarjeta muestra número, cliente, productos y estado: Pendiente, En preparación o Listo.
2. Para agregar productos, toque Agregar. Mientras el pedido está pendiente o en preparación pueden añadirse pizzas y bebidas; cuando está listo, solo bebidas.
3. Para cancelar, use Cancelar únicamente mientras el pedido está pendiente y registre un motivo. Una vez iniciada la preparación, no se cancela.
4. Cuando Cocina marque Listo, use Llamar si existe celular y luego toque Entregar.

Qué debe observar: Cocina recibe lo agregado y Recepción recibe el aviso de pedido listo. Los pedidos entregados o cancelados salen de la lista activa, pero no se eliminan físicamente.

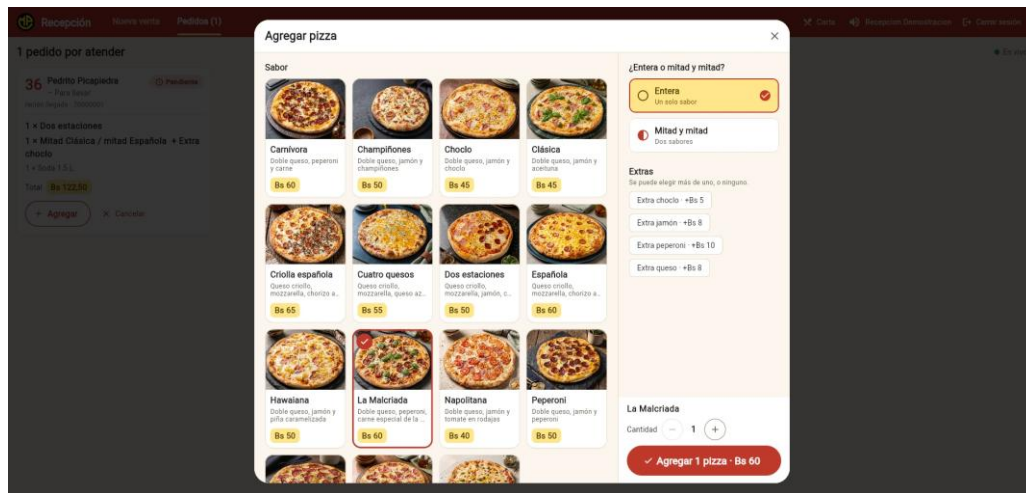


Figura A.5. Incorporación de una pizza a un pedido enviado. Fuente: captura del sistema desplegado, 08/10/2026.

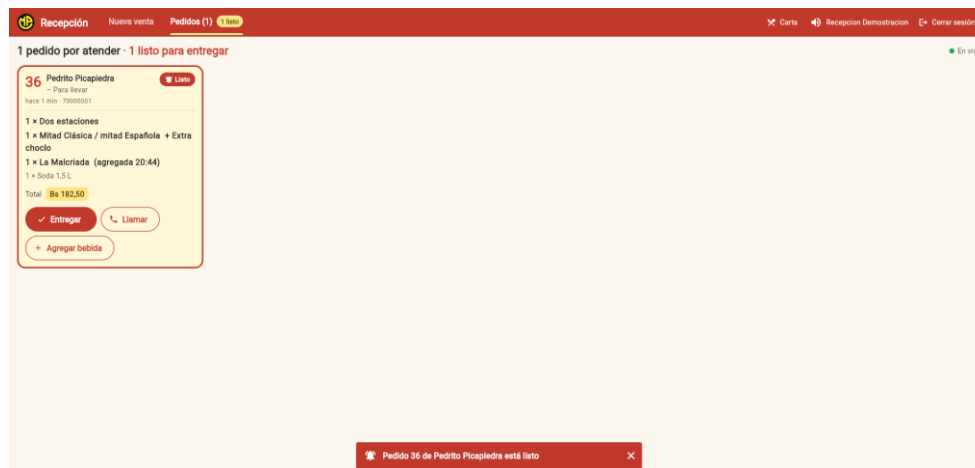


Figura A.6. Aviso de pedido listo en Recepción. Fuente: captura del sistema desplegado, 08/10/2026.

A.3 Cocina

A.3.1 Instalar y abrir la aplicación

Descargue el APK desde <https://github.com/jorgeluisgutierrezrivera/max-pizzapp/releases/latest/download/max-pizzapp-cocina.apk>, permita la instalación desde el navegador cuando Android lo solicite e ingrese con la

cuenta de Cocina. Una versión nueva se instala encima de la anterior. También puede usarse la dirección pública desde un navegador.

A.3.2 Preparar pedidos

1. Mantenga abierta la cola. Los pedidos nuevos llegan en orden y producen un aviso sonoro.
2. Toque Empezar cuando inicie la preparación. Recepción verá En preparación.
3. Toque Listo al terminar. El pedido sale de la cola y Recepción recibe el aviso.

Qué debe observar: cada tarjeta muestra las pizzas, mitades, extras, bebidas y observaciones. Lo añadido después del envío aparece identificado.

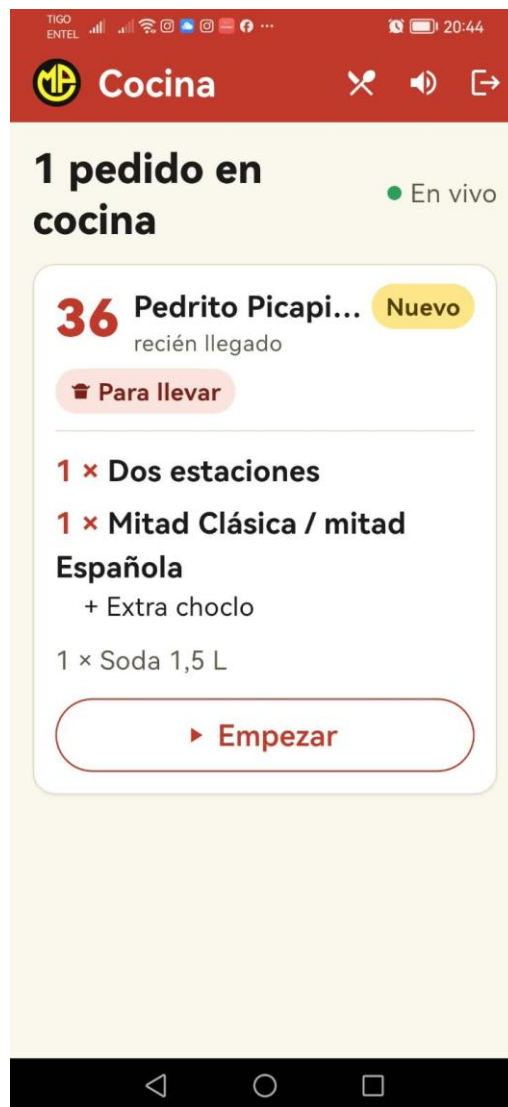


Figura A.7. Pedido nuevo en el APK de Cocina, segundo dispositivo.

Fuente: captura del sistema desplegado, 08/10/2026.

A.3.3 Administrar pizzas y extras

1. Toque Carta.
2. Cambie el interruptor de una pizza o un extra entre Disponible y Agotado.
3. Use Agotar todas o Reponer todas para una categoría completa y confirme la acción.

Qué debe observar: el cambio se refleja en Recepción y el producto deja de ofrecerse en una venta nueva.



Figura A.8. Disponibilidad de pizzas administrada desde Cocina.
Fuente: captura del sistema desplegado, 08/10/2026.

A.4 Disponibilidad por perfil

Tabla A.1. Responsabilidad sobre la disponibilidad. Fuente: reglas implementadas de RF-13 y D-76.

Perfil	Productos que puede agotar o reponer	Productos que no administra
Recepción	Bebidas	Pizzas y extras
Cocina	Pizzas y extras	Bebidas

Si se agotan todas las pizzas, Recepción ve una banda que permite continuar con Vender bebidas. Un intento de modificar la categoría del otro perfil se rechaza y no cambia la carta.

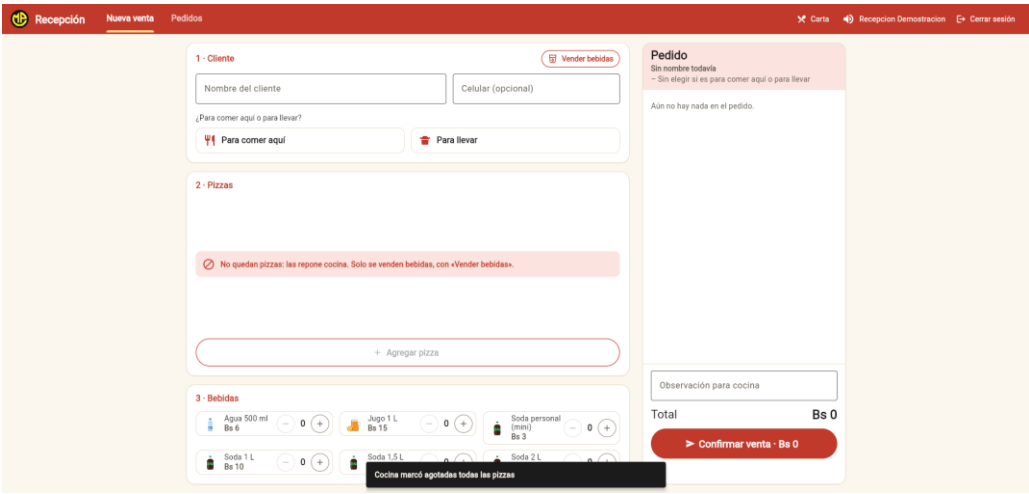


Figura A.9. Aviso en Recepción cuando Cocina agota todas las pizzas.
Fuente: captura del sistema desplegado, 08/10/2026.

A.5 Mensajes frecuentes

Tabla A.2. Mensajes de la aplicación y acción recomendada. Fuente: textos de Max Pizzapp 0.4.0.

Mensaje	Qué significa	Qué hacer
Sin conexión en vivo. Lo que ves puede no estar al día.	Se cortó el canal con el servidor.	Revise Internet; si no vuelve, toque Recargar.
El sonido está apagado.	El navegador aún no autorizó el sonido.	Toque una vez la pantalla.
Un pedido lleva al menos una pizza.	Se intentó confirmar solo bebidas como pedido.	Añada una pizza o use Vender bebidas.

Mensaje	Qué significa	Qué hacer
Producto agotado.	La disponibilidad cambió mientras se armaba la venta.	Quite o cambie el producto y confirme de nuevo.
Cocina ya empezó el pedido.	El pedido dejó de estar pendiente.	No lo cancele; resuelva el cambio con el local.
La sesión expiró.	Terminó la vigencia de la sesión.	Ingresa nuevamente; los pedidos permanecen en el servidor.
No se pudo enviar.	La venta no llegó al servidor.	Conserve la pantalla, revise la conexión y reintente.

A.6 Reglas importantes

- Solo se venden pizzas enteras: de un sabor o de dos mitades de sabores distintos.
- Dos estaciones, Tres estaciones y Criolla española se venden únicamente enteras.
- Un pedido solo puede cancelarse mientras está pendiente. Una vez iniciada la preparación, no se cancela.
- Puede agregarse contenido según el estado: pizzas y bebidas antes de estar listo; únicamente bebidas cuando ya está listo; nada después de entregado o cancelado.
- Los pedidos cancelados se conservan con su motivo; no existe una eliminación física como operación normal.

ANEXO B. MANUAL DE INSTALACIÓN Y DESPLIEGUE

Este manual separa dos procedimientos: la Parte 1 instala una instancia local en una computadora Windows para una persona no programadora; la Parte 2 describe el despliegue de producción administrado técnicamente.

Parte 1. Instalación local

B.1 Requisitos

Se necesita Windows 10 versión 22H2 o Windows 11 de 64 bits, 8 GB de memoria RAM o más, 20 GB libres, conexión a Internet y la contraseña de una cuenta administradora para preparar Windows. La primera instalación descarga Docker Desktop y cerca de 1 GB adicional.

Tabla B.1. Comprobaciones previas. Fuente: procedimiento validado el 08/10/2026.

Elemento	Dónde comprobar	Resultado requerido
Windows	Inicio > Configuración > Sistema > Información	Windows 10 22H2 o Windows 11, 64 bits
Memoria	La misma pantalla	8 GB o más
Virtualización	Administrador de tareas > Rendimiento > CPU	Virtualización: Habilitado
Disco	Explorador > Este equipo	20 GB libres o más

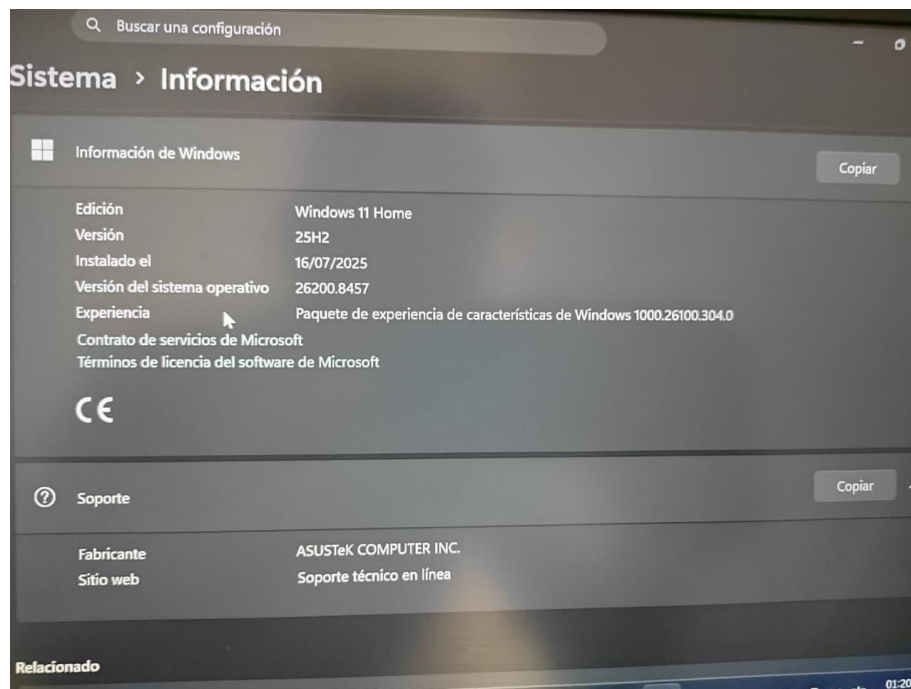


Figura B.1. Verificación de Windows 11 Home 25H2 en el equipo de prueba. Fuente: fotografía de la prueba externa, 08/10/2026.

B.2 Preparar Windows e instalar Docker Desktop

1. Haga clic derecho en Inicio y abra Terminal (Administrador). Escriba `wsl --install --no-distribution`, presione Enter y reinicie la computadora.
2. Entre a <https://www.docker.com/products/docker-desktop/>, descargue Docker Desktop para Windows AMD64 y abra el instalador.
3. Deje seleccionadas Per-user installation y Add shortcut to desktop. Termine la instalación y abra Docker Desktop.
4. Acepte el acuerdo, continúe sin iniciar sesión y espere a que aparezca Engine running.

Qué debe observar: Docker queda en ejecución. Si aparece Virtual Machine Platform not enabled, repita el paso de preparación de Windows y reinicie; reiniciar solo Docker no basta.

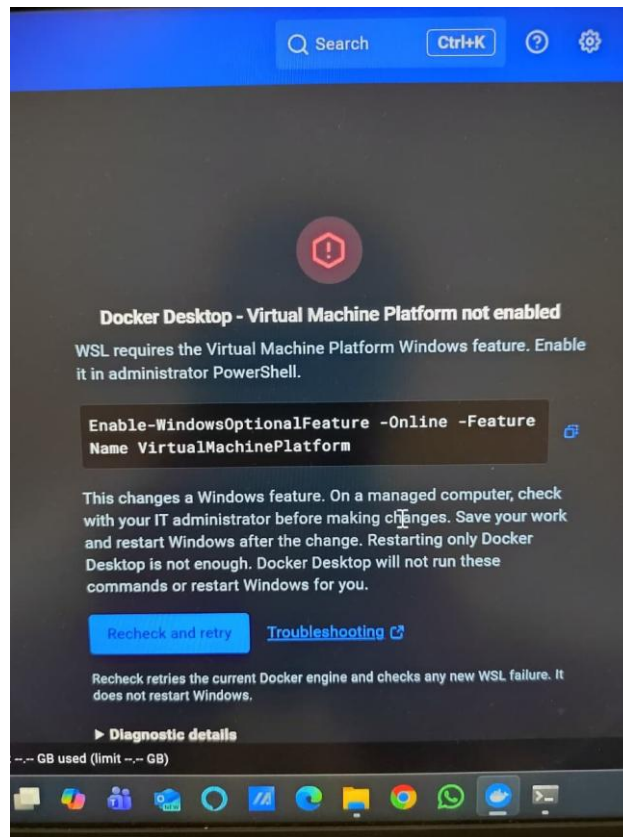


Figura B.2. Incidencia de Virtual Machine Platform detectada en la prueba. Fuente: fotografía de la prueba externa, 08/10/2026.

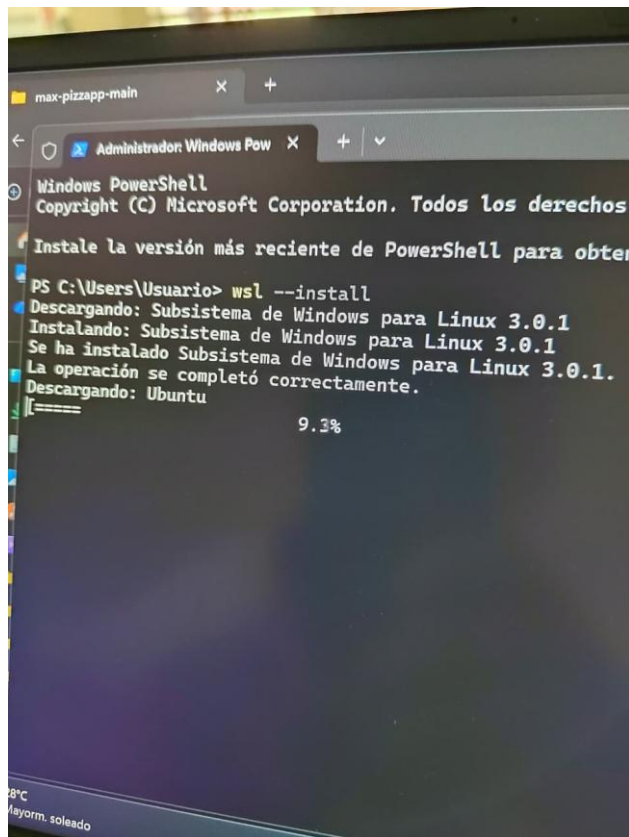


Figura B.3. Instalación de WSL aplicada durante la prueba. Fuente: fotografía de la prueba externa, 08/10/2026.

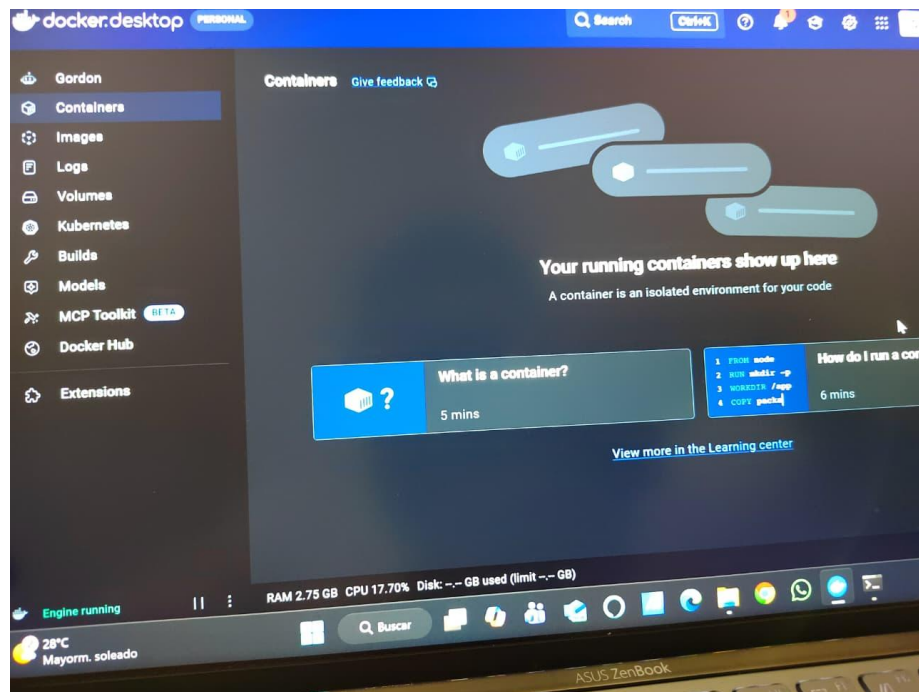


Figura B.4. Docker Desktop con el motor en ejecución. Fuente: fotografía de la prueba externa, 08/10/2026.

B.3 Descargar Max Pizzapp

1. Abra <https://github.com/jorgeluisgutierrezrivera/max-pizzapp>.
2. Toque Code > Download ZIP.
3. En Descargas, use Extraer todo. Abra la carpeta extraída y luego la carpeta interna con el mismo nombre.

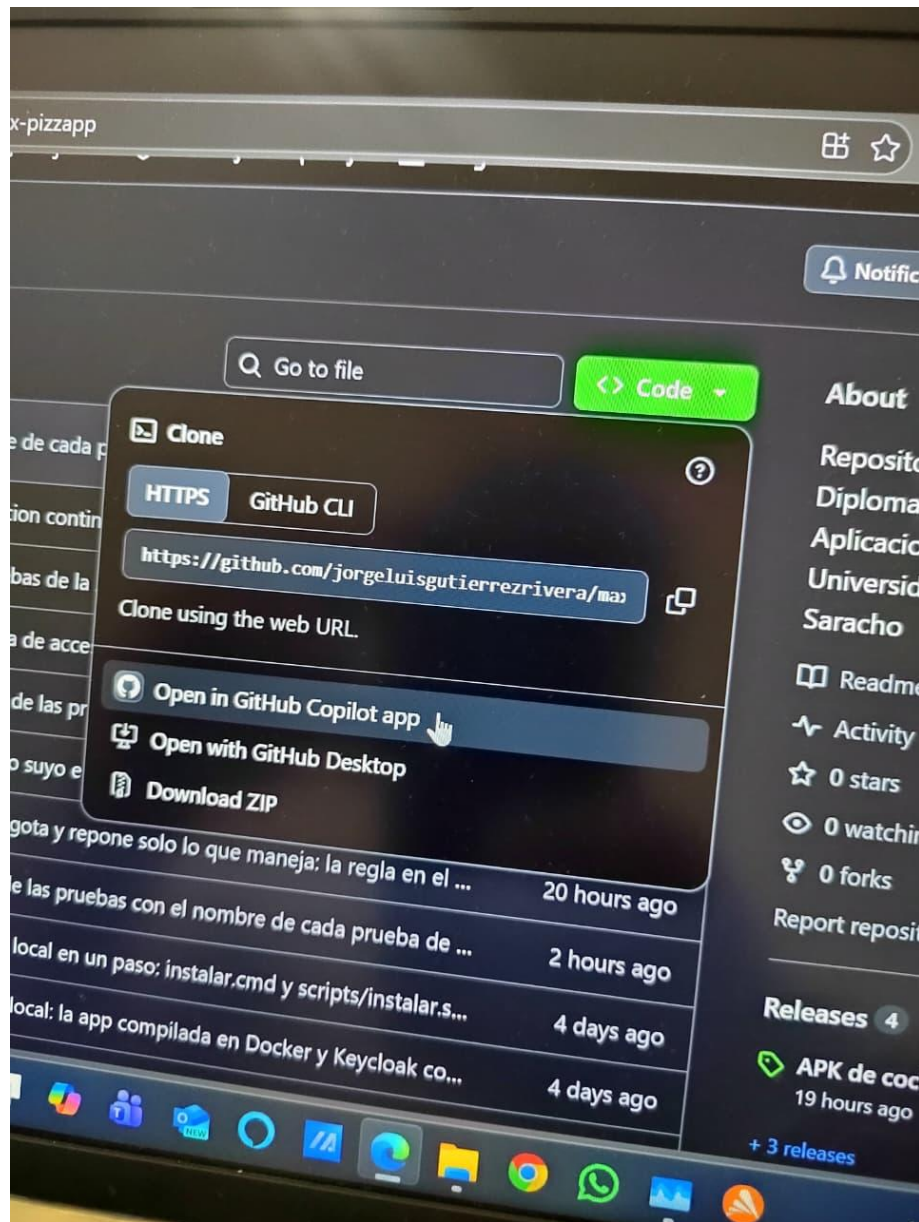
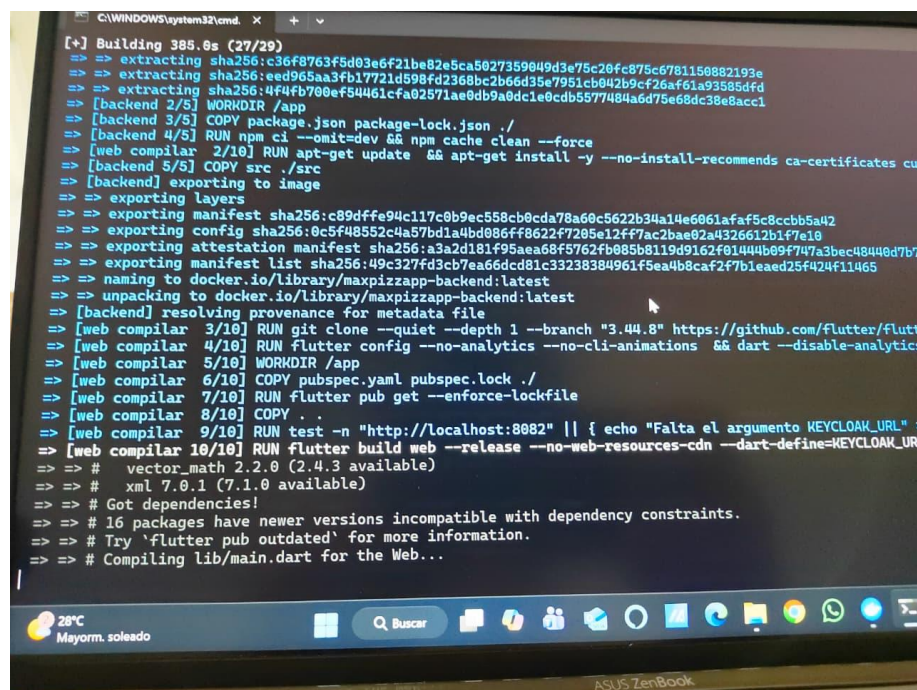


Figura B.5. Descarga del archivo ZIP del proyecto. Fuente: fotografía de la prueba externa, 08/10/2026.

B.4 Instalar el sistema

1. Compruebe que Docker Desktop diga Engine running.
2. Haga doble clic en instalar.cmd. Si Windows muestra una advertencia, use Más información > Ejecutar de todas formas.

3. No cierre la ventana. El instalador avanza del paso [1/6] al [6/6]; la construcción del paso [4/6] es la más lenta.
4. Si el Firewall pregunta por Docker Desktop Backend, elija Cancelar. La instalación local solo se usa en esa computadora.
5. Cuando aparezca [6/6] Listo, anote la contraseña generada y conserve la dirección <http://localhost:8090>. Puede volver a abrir `instalar.cmd` para ver nuevamente esos datos.



```
[*] Building 385.0s (27/29)
=> extracting sha256:c36f8763f5d03e6f21be82e5ea5027359040d3e75c70fc875c6781150882193e
=> extracting sha256:eed965aa3fb17721d598fd2368bc2b66d35e7951cb042b9cf26af61a93585d5fd
=> extracting sha256:4f4fb700ef54461cfa02571ae0db9a0dc1e0c0b5577484a6d75e68dc38e8acc1
=> [backend 2/5] WORKDIR /app
=> [backend 3/5] COPY package-lock.json package-lock.json ./
=> [backend 4/5] RUN npm ci --omit=dev && npm cache clean --force
=> [web compiler 2/10] RUN apt-get update && apt-get install -y --no-install-recommends ca-certificates curl
=> [backend 5/5] COPY src ./src
=> [backend] exporting to image
=> exporting layers
=> exporting manifest sha256:c89dffe94c117c0b9ec558cb0cda78a60c5622b34a14e6061afaf5c8cbb5a42
=> exporting config sha256:0c5f48552c4a57bd1a4bd086ff8622f7205e12ff7ac2bae02a4326612b1f7e10
=> exporting attestation manifest sha256:a3a2d181f95aea68f5762fb085b8119d9162f01444b09f747a3bec48440d7b75
=> exporting manifest list sha256:49c327fd3cb7ea66dcd81c33238384961f5ea4b8caf2f7b1eaed25f424f11465
=> naming to docker.io/library/maxpizzapp-backend:latest
=> unpacking to docker.io/library/maxpizzapp-backend:latest
=> [backend] resolving provenance for metadata file
=> [web compiler 3/10] RUN git clone --quiet --depth 1 --branch "3.44.8" https://github.com/flutter/flutter
=> [web compiler 4/10] RUN flutter config --no-analytics --no-cli-animations && dart --disable-analytics
=> [web compiler 5/10] WORKDIR /app
=> [web compiler 6/10] COPY pubspec.yaml pubspec.lock ./
=> [web compiler 7/10] RUN flutter pub get --enforce-lockfile
=> [web compiler 8/10] COPY . .
=> [web compiler 9/10] RUN test -n "http://localhost:8082" || { echo "Falta el argumento KEYCLOAK_URL" && exit 1 }
=> [web compiler 10/10] RUN flutter build web --release --no-web-resources-cdn --dart-define=KEYCLOAK_URL=http://localhost:8082
=> # vector_math 2.2.0 (2.4.3 available)
=> # xml 7.0.1 (7.1.0 available)
=> # Got dependencies!
=> # 16 packages have newer versions incompatible with dependency constraints.
=> # Try 'flutter pub outdated' for more information.
=> # Compiling lib/main.dart for the Web...
```

Figura B.6. Construcción de los componentes durante el paso [4/6].

Fuente: fotografía de la prueba externa, 08/10/2026.

1. En la ventana abierta por el instalador, ingrese con recepcion.demo y la contraseña generada.
2. Registre una venta con datos inventados y confirme.
3. Abra una segunda ventana privada: Ctrl + Mayús + N en Chrome o InPrivate en Edge. Entre a <http://localhost:8090> con cocina.demo y la misma contraseña.
4. En Cocina toque Empezar y luego Listo. Regrese a Recepción y toque Entregar.

Qué debe observar: el pedido aparece en Cocina, cambia de estado y vuelve a Recepción como listo. Si el recorrido termina, la instancia local quedó funcional.

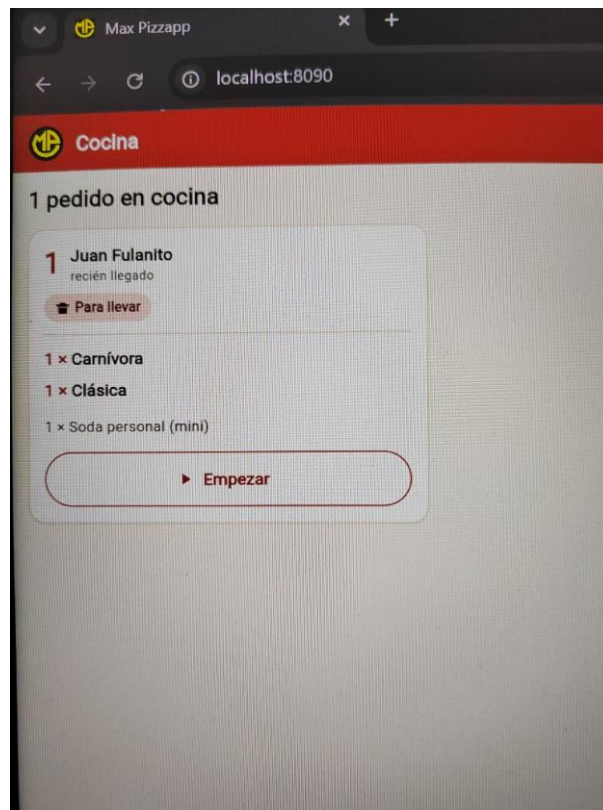


Figura B.9. Pedido de prueba recibido en Cocina. Fuente: fotografía de la prueba externa, 08/10/2026.

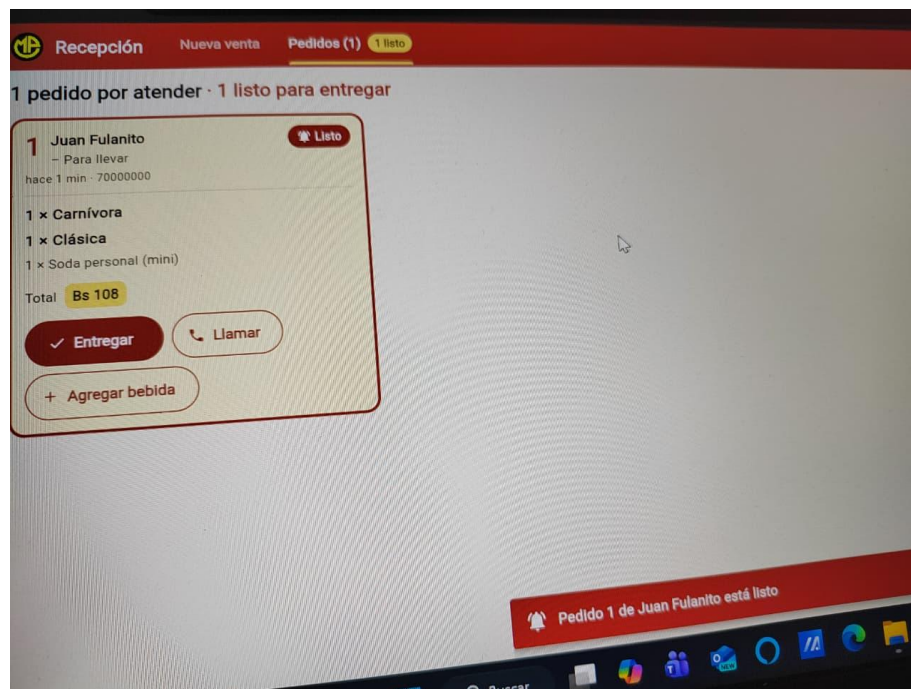


Figura B.10. Pedido listo para entregar en Recepción. Fuente: fotografía de la prueba externa, 08/10/2026.

B.6 Detener y volver a abrir

Para detener la instancia, abra Docker Desktop > Containers, localice maxpizzapp y use Stop. Para volver a abrirla, espere a que Docker indique Engine running y ejecute instalar.cmd de nuevo; los pedidos se conservan.

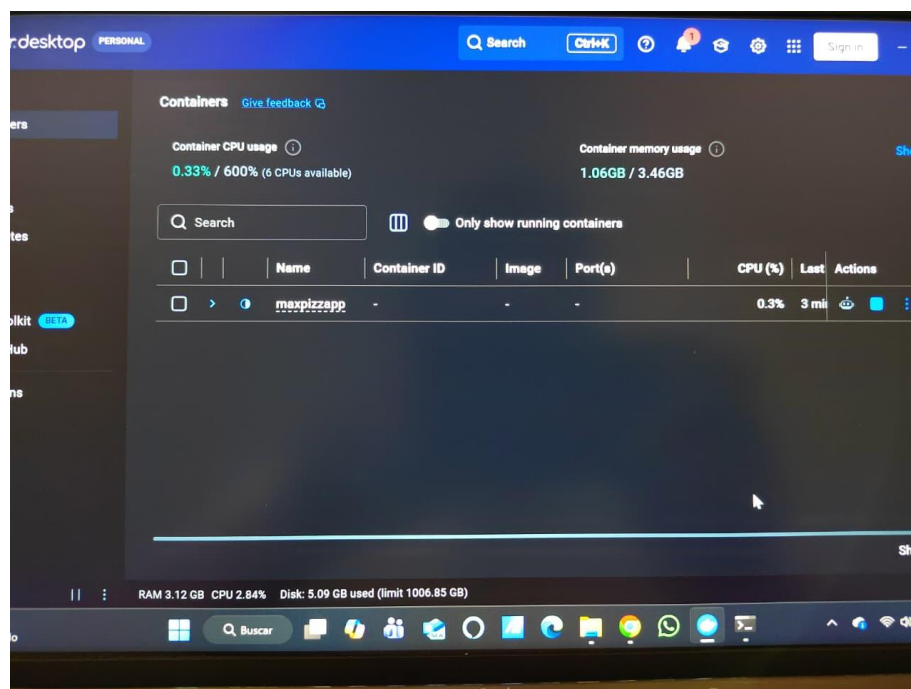


Figura B.11. Contenedores de Max Pizzapp en funcionamiento. Fuente: fotografía de la prueba externa, 08/10/2026.

B.7 Problemas frecuentes

Tabla B.2. Incidencias y solución documentada. Fuente: manual corregido tras la prueba externa.

Situación	Acción
Virtual Machine Platform not enabled	Ejecute <code>wsl --install --no-distribution</code> como administrador y reinicie Windows.
Docker Desktop pide actualizar WSL	En Terminal (Administrador), ejecute <code>wsl --update</code> y vuelva a abrir Docker.
Firewall pregunta por Docker Desktop Backend	Elija Cancelar; los puertos están limitados a 127.0.0.1.
Docker no está abierto	Abra Docker Desktop, espere Engine running y repita <code>instalar.cmd</code> .
El puerto 8090 está ocupado	Cierre el programa indicado o reinicie la computadora y vuelva a intentar.
No encuentra la contraseña	Ejecute <code>instalar.cmd</code> otra vez; la última línea vuelve a mostrarla.

Situación	Acción
Cocina abre la sesión de Recepción	Use una ventana privada o InPrivate para la segunda cuenta.

B.8 Validación externa del procedimiento

El 8 de octubre de 2026, el encargado de recepción siguió el manual impreso en la computadora nueva del local, con Windows 11 Home 25H2. La ejecución se registró de 13:00 a 14:30 e incluyó Docker Desktop, la instalación y el flujo funcional. El sistema levantó; la salud respondió correctamente y el pedido recorrió venta, Cocina, estado listo y entrega.

Se registraron tres intervenciones del autor: resolver la preparación de WSL, indicar el uso de una ventana privada y apoyar el ingreso con la contraseña de prueba. Estas incidencias permitieron ajustar el manual. Durante la prueba se permitió inicialmente el acceso del Firewall; después de la prueba, la configuración local se restringió a 127.0.0.1 y el manual se actualizó para indicar Cancelar. La prueba no demuestra una instalación completamente autónoma, pero sí que otra persona siguió el procedimiento y que el sistema funcionó al finalizar.

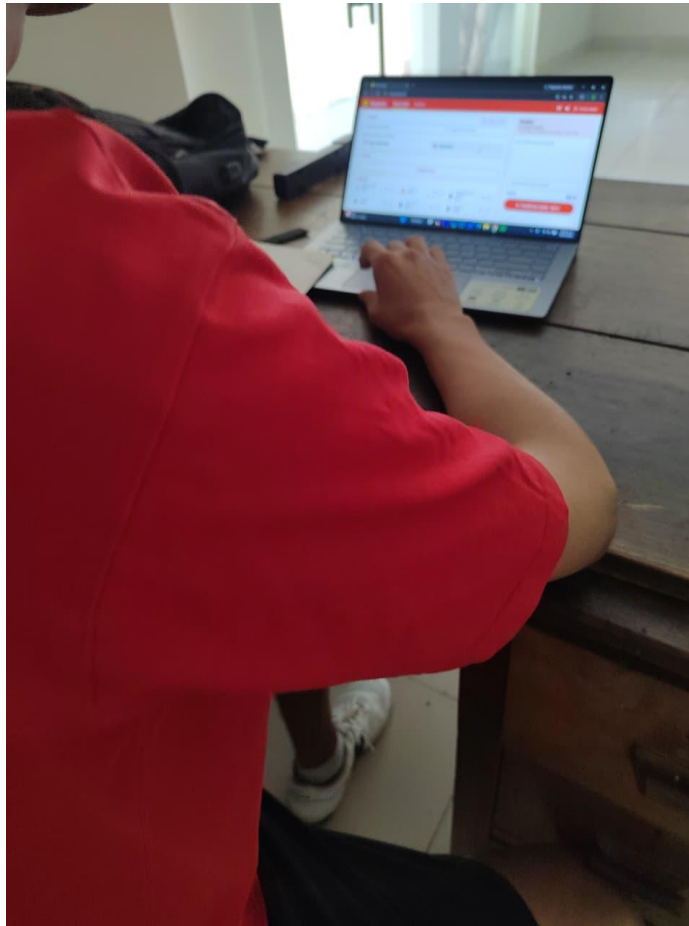


Figura B.12. Encargado de recepción durante la prueba externa. Fuente: fotografía autorizada, 08/10/2026.

Parte 2. Despliegue de producción

B.9 Requisitos del servidor

Se requiere un servidor Linux con Docker y Compose, al menos 2 GB de memoria, los puertos TCP 22, 80 y 443 y UDP 443 habilitados, y un dominio con registros para el dominio raíz y auth antes del primer arranque. Esta parte corresponde a una persona con administración técnica del servidor.

B.10 Primer despliegue

1. Ubique el código en /opt/maxpizzapp.
2. Cree el archivo .env en el servidor con permisos 600 y contraseñas generadas allí. No copie secretos al repositorio ni al documento.
3. Valide y levante docker/docker-compose.prod.yml. Caddy solicitará los certificados en el primer arranque.
4. Aplique el endurecimiento de Keycloak con el procedimiento versionado y reemplace la cuenta administrativa temporal.
5. Publique la aplicación web desde la máquina de desarrollo con el procedimiento documentado en el README.

B.11 Verificar producción

Tabla B.3. Comprobaciones externas del despliegue. Fuente: README y reportes de producción.

Comprobación	Resultado esperado
https://maxpizzapp.tech/	200 por HTTPS
http://maxpizzapp.tech/	308 hacia HTTPS
/api/v1/salud	200 con estado y base de datos correctos
Emisor de Keycloak	https://auth.maxpizzapp.tech/realms/maxpizzapp
Puertos externos	Solo 22, 80 y 443
Cuentas de prueba	Ingreso por PKCE y flujo de venta a entrega

B.12 Actualizar y distribuir el APK

Las actualizaciones del servidor se aplican fuera del horario de atención. Si existe una migración, se respalda la base antes de ejecutarla. La aplicación de Cocina se compila con la llave conservada fuera del repositorio y se publica como APK con su huella SHA-256. El enlace estable de descarga es <https://github.com/jorgeluisgutierrezrivera/max-pizzapp/releases/latest/download/max-pizzapp-cocina.apk>.

ANEXO C. ENLACE AL REPOSITORIO DE CÓDIGO

Repositorio público del proyecto:

<https://github.com/jorgeluisgutierrezrivera/max-pizzapp>

El repositorio contiene el código fuente, el contrato OpenAPI, los archivos de despliegue, los scripts de verificación, los reportes de prueba y la documentación técnica. Este enlace se proporciona para consulta y trazabilidad del artefacto de software.

ANEXO D. ACCESO PÚBLICO O INSTALABLE

Recurso	Dirección
Aplicación web	https://maxpizzapp.tech
Salud de la API	https://maxpizzapp.tech/api/v1/salud
Proveedor de identidad	https://auth.maxpizzapp.tech
APK de Cocina 0.4.0	https://github.com/jorgeluisgutierrezrivera/max-pizzapp/releases/latest/download/max-pizzapp-cocina.apk
Estado público	https://stats.uptimerobot.com/AvecPe8ITs

Tabla D.1. Accesos públicos e instalables. Fuente: verificación del autor, 07/10/2026.

No se incluyen credenciales, contraseñas, tokens ni valores del archivo de entorno.

ANEXO E. DIAGRAMAS EN TAMAÑO LEGIBLE

Los diagramas siguientes son los mismos citados en el cuerpo y se reproducen a mayor tamaño para facilitar su lectura.

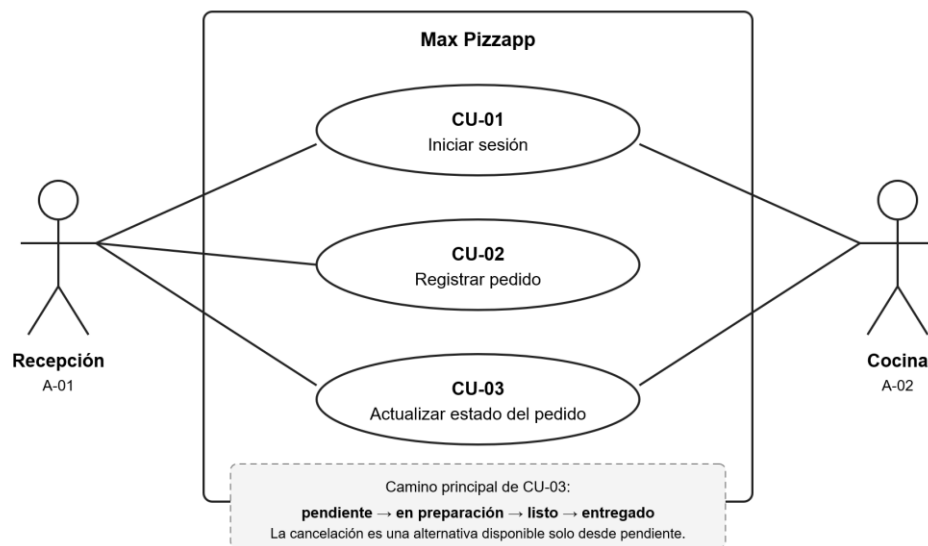


Figura E.1. Diagrama de casos de uso. Fuente: elaboración propia.

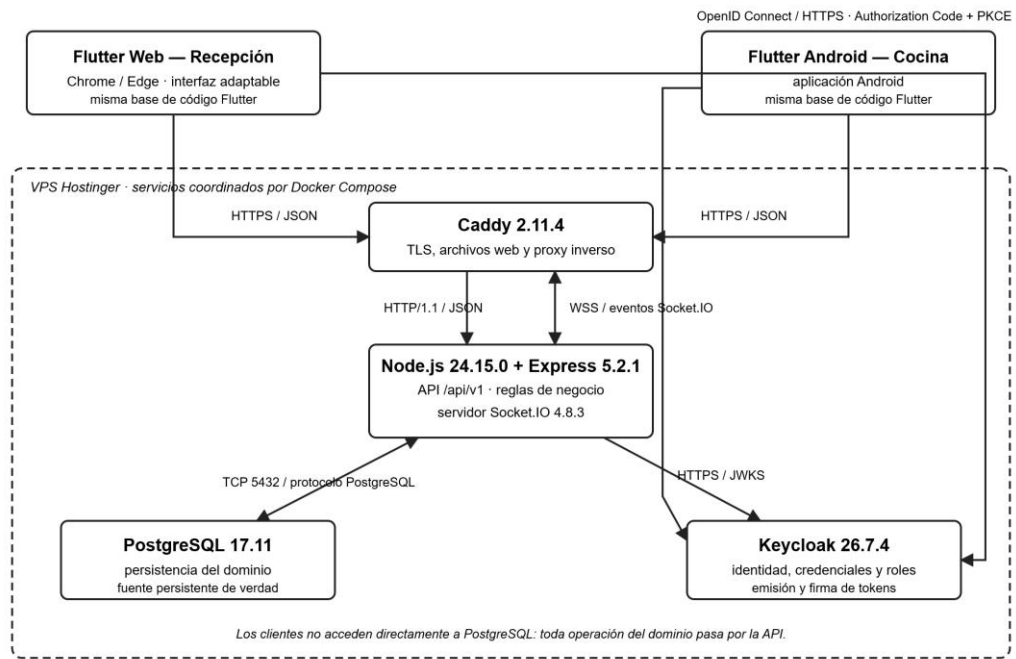


Figura E.2. Arquitectura actual. Fuente: elaboración propia.

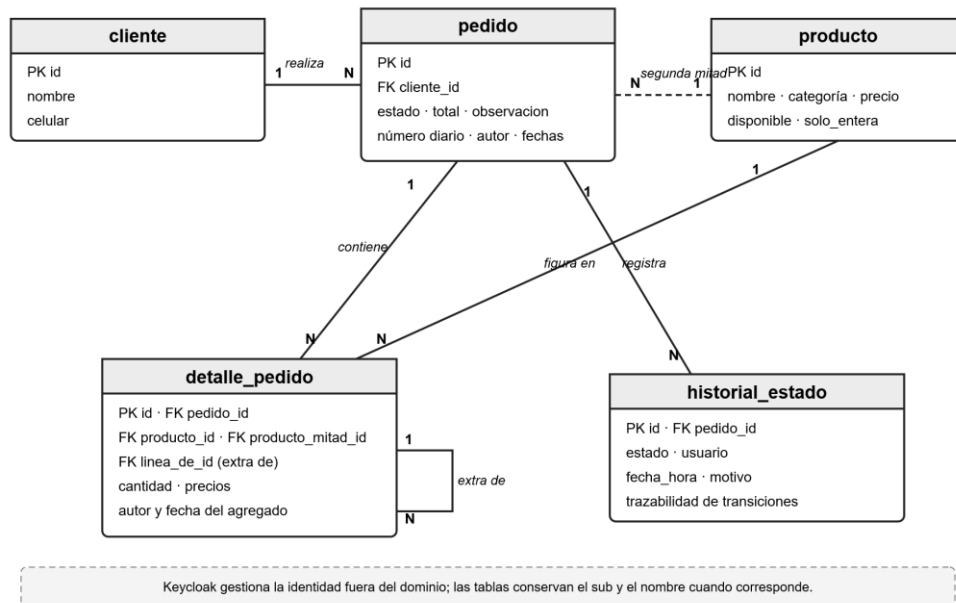


Figura E.3. Modelo entidad-relación. Fuente: elaboración propia.

Recepción — web, escritorio 1366 px

Max Pizzapp · Recepción

Nueva venta | Pedidos | Cerrar sesión

1 · Cliente

Nombre · celular opcional · local / para llevar

2 · Pizzas

Agregar pizza entera o mitad y mitad

Elegir sabores y extras

[+ Agregar otra pizza]

3 · Bebidas

Bebidas con cantidad

Pedido actual

Productos y cantidades

Observación para cocina

Total

[Confirmar venta]

La composición de Recepción prioriza el formulario completo en computadora. Cocina conserva botones grandes, estado textual y cola legible. Los estados cargando, con datos, vacío y error se resuelven en el mismo contenedor de cada vista.

Cocina — Android

Cocina

En vivo

2 pedidos en cocina

#44 · Ana · Para llevar

Mitad Choclo / mitad Española

Observación: sin cebolla

[Empezar]

#43 · Luis · En preparación

Pizza Pepperoni + extra

Agregado recientemente

15 min · demorado

[Listo]

Mismo módulo en tableta

pedido #44

pedido #43

Prototipos de baja fidelidad elaborados a partir de la interfaz vigente; no contienen datos reales de clientes.

Figura E.4. Prototipos de las pantallas. Fuente: elaboración propia.

ANEXO F. DOCUMENTACIÓN COMPLEMENTARIA

F.1 Reglas de negocio

Regla	Aplicación en Max Pizzapp
Pizza entera	Solo se venden pizzas enteras: de un sabor o compuestas por dos mitades de sabores distintos.
Dos sabores	El precio es (precio A + precio B) / 2. El servidor efectúa el cálculo final y contrasta el total esperado.
Solo sabor único	Dos estaciones, Tres estaciones y Criolla española se venden enteras y no pueden utilizarse como mitad.
Cancelación	Es una baja lógica y solo procede mientras el pedido está pendiente. No existe DELETE como operación normal del pedido.
Agregados	Pendiente o en preparación: pizzas y bebidas; listo: solo bebidas; entregado o cancelado: ninguno. Las líneas ya enviadas permanecen sin modificación ni eliminación.
Venta directa	Una venta compuesta solo por bebidas nace entregada y no pasa por Cocina.
Datos de carta	Los precios de las 15 pizzas y de las 6 bebidas están confirmados con información del negocio. Los extras conservan valores provisionales hasta contar con los datos definitivos.

Tabla F.1. Reglas de negocio vigentes. Fuente: decisiones y esquema del proyecto.

F.2 Componentes de la arquitectura

Componente	Responsabilidad	Tecnología	Se comunica con	Protocolo / formato
Cliente web	Toma y seguimiento de pedidos de Recepción; conserva técnicamente las vistas compartidas.	Flutter 3.44.8 / Dart 3.12.2	Caddy, API, Socket.IO y Keycloak	HTTPS/JSON; WSS/eventos; OIDC/HTTPS
Cliente Android	Cola y actualización de pedidos de Cocina desde el segundo destino Flutter.	Flutter 3.44.8, flutter_appauth 12.1.0	Caddy, API, Socket.IO y Keycloak	HTTPS/JSON; WSS/eventos; OIDC/PKCE
Proxy inverso	Termina TLS, sirve la web y enruta API, Socket.IO e identidad.	Caddy 2.11.4	Cientes, API y Keycloak	HTTPS público; HTTP/WS internos
Servidor de aplicación	Aplica roles, transiciones, precios y persistencia; expone el contrato HTTP.	Node.js 24.15.0 + Express 5.2.1	Caddy, PostgreSQL, Keycloak	HTTP/JSON; TCP PostgreSQL; HTTPS/JWKS

Componente	Responsabilidad	Tecnología	Se comunica con	Protocolo / formato
Canal de eventos	Notifica pedidos, agregados, estados y recuperación del canal; forma parte del proceso Node.	Socket.IO 4.8.3	Clientes web y Android	WSS / eventos Socket.IO
Base de datos	Conserva pedidos, líneas, carta e historial como fuente persistente de verdad.	PostgreSQL 17.11	Servidor de aplicación y en Keycloak bases separadas	TCP 5432 / PostgreSQL
Proveedor de identidad	Autentica, administra credenciales y roles, y firma tokens.	Keycloak 26.7.4	Clientes, API y PostgreSQL	OpenID Connect/HTTPS; JWKS/JSON; SQL interno

Tabla F.2. Componentes y responsabilidades. Fuente: configuración vigente.

F.3 Diccionario de cliente

Campo	Tipo	Longitud	Obligatorio	Restricción	Descripción
id	serial	—	Sí	PK	Identificador interno.
nombre	varchar	120	Sí	No vacío	Nombre comunicado por Recepción.
celular	varchar	20	No	Único si existe; ^[5-7][0-9]{7}\$	Número boliviano opcional: 8 dígitos que empiezan con 5, 6 o 7.

Tabla F.3. Diccionario de datos de cliente. Fuente: esquema vigente.

F.4 Diccionario de pedido

Campo	Tipo	Longitud	Obligatorio	Restricción	Descripción
id	serial	—	Sí	PK	Identificador interno.
cliente_id	integer	—	Condicional	FK cliente; RESTRICT	Nulo solo en venta directa.
numero_del_dia	integer	—	Condicional	Único con día; > 0	Número mostrado; nulo en venta directa.
dia	date	—	Sí	Generado desde creado_en	Día de operación en hora boliviana.
para_llevar	boolean	—	Condicional	Consistencia pedido/venta directa	Modalidad del pedido.

Campo	Tipo	Longitud	Obligatorio	Restricción	Descripción
creado_por	text	—	Sí	No vacío	sub de Keycloak que crea.
creado_por_nombre	varchar	120	Sí	—	Nombre copiado al crear.
estado	estado_pedido	—	Sí	Enum; pendiente por defecto	Estado lógico.
observacion	varchar	240	No	—	Indicación para Cocina.
total	numeric	10,2	Sí	>= 0	Total persistido.
creado_en	timestampz	—	Sí	now()	Fecha y hora de creación.

Tabla F.4. Diccionario de datos de pedido. Fuente: esquema vigente.

F.5 Diccionario de detalle_pedido

Campo	Tipo	Longitud	Obligatorio	Restricción	Descripción
id	serial	—	Sí	PK; único con pedido_id	Identificador de línea.
pedido_id	integer	—	Sí	FK pedido; CASCADE	Pedido propietario.
producto_id	integer	—	Sí	FK producto; RESTRICT	Producto principal o extra.
producto_mitad_id	integer	—	No	FK producto; distinto del principal	Segundo sabor de una pizza.
linea_de_id	integer	—	No	FK compuesta al detalle del mismo pedido	Línea de pizza a la que pertenece un extra.
cantidad	integer	—	Sí	1 a 999	Unidades.
precio_unitario	numeric	10,2	Sí	>= 0	Precio guardado al vender.
subtotal	numeric	10,2	Sí	>= 0	Subtotal de la línea.
agregado_en	timestampz	—	No	Completo con autor y nombre	Momento de un agregado posterior.
agregado_por	text	—	No	Completo con fecha y nombre	sub que agregó la línea.

Campo	Tipo	Longitud	Obligatorio	Restricción	Descripción
agregado_por_nombre	varchar	120	No	Completo con fecha y sub	Nombre copiado al agregar.

Tabla F.5. Diccionario de datos de detalle_pedido. Fuente: esquema vigente.

F.6 Diccionario de producto

Campo	Tipo	Longitud	Obligatorio	Restricción	Descripción
id	serial	—	Sí	PK	Identificador.
nombre	varchar	120	Sí	Único; no vacío	Nombre de carta.
categoria	categoria_producto	—	Sí	Enum	Pizza, entrada, bebida, postre o extra.
precio	numeric	10,2	Sí	≥ 0	Precio de venta.
disponible	boolean	—	Sí	true por defecto	Disponibilidad simple.
descripcion	varchar	200	No	No vacía si existe	Ingredientes o detalle.
imagen	varchar	80	No	Solo nombre de archivo	Recurso visual.
solo_entera	boolean	—	Sí	Solo aplicable a pizza	Impide usarla como mitad.
creado_en	timestampz	—	Sí	now()	Alta del registro.

Tabla F.6. Diccionario de datos de producto. Fuente: esquema vigente.

F.7 Diccionario de historial de estado

Campo	Tipo	Longitud	Obligatorio	Restricción	Descripción
id	serial	—	Sí	PK	Identificador.
pedido_id	integer	—	Sí	FK pedido; CASCADE	Pedido trazado.
estado	estado_pedido	—	Sí	Enum	Estado registrado.
usuario_id	text	—	Sí	No vacío	sub que ejecutó el cambio.
usuario_nombre	varchar	120	Sí	—	Nombre copiado en el momento.

Campo	Tipo	Longitud	Obligatorio	Restricción	Descripción
fecha_hora	timestampz	—	Sí	now()	Instante del cambio.
motivo	varchar	120	Condicional	Obligatorio solo al cancelar	Causa de baja lógica.

Tabla F.7. Diccionario de datos de historial_estado. Fuente: esquema vigente.

F.8 Tarjetas del desarrollo

Tarjeta	Contenido	Requisitos	Estado
01–07	Base de datos, API, web de Recepción, cola de Cocina, estados, tiempo real y seguridad inicial	RF-01–RF-12	Cerradas
08	Disponibilidad de productos y autorización por categoría	RF-13	Cerrada
09–12	Seguridad, pruebas, despliegue y APK de Cocina	RNF y E2/E3	Cerradas
13	Instalación y documentación operativa	Objetivo 6	Cerrada; prueba externa completada el 08/10/2026
14	Bebidas reales y venta solo de bebidas	RF-02, RF-03 y RF-13	Cerrada

Tabla F.8. Tarjetas e incrementos al 8 de octubre de 2026. Fuente: tablero, evidencias y planes del proyecto.

F.9 Matriz OWASP Top 10:2025

Categoría	Tratamiento en Max Pizzapp
A01 — Pérdida de Control de Acceso	Token y rol verificados en el servidor en cada ruta; transiciones por rol; cuentas nuevas sin rol (403); Cocina no recibe el celular del cliente.
A02 — Configuración de Seguridad Incorrecta	Cabeceras de seguridad con Helmet; CSP impuesta; la API no anuncia su tecnología; Keycloak configurado por script. La base de datos y los servicios internos no se exponen directamente a Internet; el acceso público se concentra en los servicios necesarios para la solución.
A03 — Fallas en la Cadena de Suministro de Software	Dependencias exactas con package-lock.json y pubspec.lock; npm audit sin vulnerabilidades; versiones fijadas de cada imagen; ningún script de terceros en la app.
A04 — Fallas Criptográficas	HTTPS con HSTS en todo el tráfico público y WSS en el canal; contraseñas con Argon2id; tokens firmados, verificados con las claves públicas del emisor.
A05 — Inyección	Consultas parametrizadas, sin concatenar texto del usuario en SQL; los textos se muestran sin interpretar HTML; caracteres de control rechazados.
A06 — Diseño Inseguro	El precio lo decide el servidor; las reglas del negocio viven en el servidor y en la base; la versión del pedido evita decisiones sobre datos que no se vieron.
A07 — Fallas de Autenticación	Keycloak con OpenID Connect y PKCE; bloqueo tras cinco intentos; contraseñas de 12 caracteres o más; token de 60 minutos.
A08 — Fallas en la Integridad del Software o de los Datos	Transacciones en cada operación; el aviso en vivo solo después del COMMIT; migraciones numeradas que no se editan; las pruebas corren en cada push.
A09 — Fallas en el Registro, Alerta y Monitoreo de Seguridad	Eventos de acceso de Keycloak durante 7 días; historial de cada pedido con quién, cuándo y por qué; monitor de disponibilidad con aviso por correo.
A10 — Manejo de Inadecuado de Condiciones Excepcionales	Ningún dato de entrada hace responder 500: 400, 413 o 415 en el formato único; 429 pasadas las 600 peticiones por minuto de una IP.

Tabla F.9. Riesgos OWASP Top 10:2025 y tratamiento en Max Pizzapp. Fuente: OWASP Foundation (2025) y controles verificados.

F.10 Configuración del canal en vivo

```
const LATIDO = { pingInterval: 4000, pingTimeout: 3000 };
io.on('connection', (socket) => {
  // El servidor une el cliente a la sala de su rol
  // y corta la conexión cuando vence el token.
});
```

Figura F.1. Configuración resumida del latido y del límite de sesión. Fuente: backend/src/tiempo-real.js.

F.11 Matriz completa de casos de prueba

ID	Criterio	Escenario	Esperado	Obtenido	Estado
CP-01	CA-01.1	Credenciales válidas en Keycloak	Token con rol y acceso a la pantalla correspondiente	Dos cuentas reconocidas, rol y vigencia de 60 min	Aprobado
CP-02	CA-01.2	Contraseña equivocada	Sin código ni token	Credenciales rechazadas	Aprobado
CP-03	CA-02.1	Venta de productos disponibles	Pedido pendiente, número diario y precio del servidor	201; pendiente; Bs 193 (95 + 58 + 2 × 20); número diario	Aprobado
CP-04	CA-02.2	Venta sin productos	No crear y solicitar un producto	400 VENTA_INVALIDA; la app impide confirmar	Aprobado
CP-05	CA-03.1	Cocina cambia el estado	Recepción lo ve en menos de 2 s	Máximo 450 ms; mediana 155 ms; 30 repeticiones	Aprobado
CP-06	CA-03.2	Pérdida del canal en vivo	Aviso y opción Recargar	Banda al segundo; Recargar consulta la API	Aprobado
CP-07	CA-04.1	Cocina marca listo	Aviso visible y sonoro en Recepción	Aviso, contador y título del navegador	Aprobado
CP-08	CA-04.2	No hay pedidos listos	Estado vacío sin avisos pendientes	Mensaje de ausencia de pedidos	Aprobado
CP-09	CA-05.1	Recepción entrega uno listo	Pasa a entregado y sale de la lista	200; entregado; historial de cuatro pasos	Aprobado
CP-10	CA-05.2	Recepción intenta entregar uno no listo	Rechazo de la transición	409	Aprobado
CP-11	CA-06.1	Recepción confirma un pedido	Aparece al final de la cola en menos de 2 s	Máximo 162 ms; mediana 156 ms; marcado Nuevo	Aprobado
CP-12	CA-06.2	No hay pedidos pendientes	Cocina muestra estado vacío	Mensaje de ausencia de pedidos	Aprobado
CP-13	CA-07.1	Cocina inicia y marca listo	De pendiente a en preparación y luego a listo	200 y 200; autor de cada cambio	Aprobado
CP-14	CA-07.2	Cocina cambia un pedido entregado	Rechazo por estado final	409	Aprobado
CP-15	Seguridad	Ruta protegida sin token	401	401 TOKEN_AUSENTE	Aprobado

ID	Criterio	Escenario	Esperado	Obtenido	Estado
CP-16	Seguridad	Token con firma alterada	401	401 TOKEN_INVALIDO	Aprobado
CP-17	Seguridad	Cocina intenta vender, cancelar o agregar	403	403 ROL_SIN_PERMISO en los tres casos	Aprobado
CP-18	Seguridad	Recepción intenta iniciar preparación	403	403 ROL_SIN_PERMISO	Aprobado
CP-19	Seguridad	Cuenta nueva sin rol	403 en rutas protegidas	403 ROL_SIN_PERMISO	Aprobado
CP-20	Seguridad	Datos inválidos	400 con formato uniforme	400 con código específico	Aprobado
CP-21	Seguridad	Cuerpo de 150 KB o codificación no admitida	413 / 415	413 y 415	Aprobado
CP-22	Seguridad	Más de 600 solicitudes por minuto e IP	429 desde la solicitud 601	600 respuestas normales y 100 con 429	Aprobado
CP-23	RNF-01	30 propagaciones por cada evento	Peor caso menor a 2 s	Máximos: 162, 450, 166 y 315 ms	Aprobado
CP-24	RNF-01	Listado con 50 pedidos y 5 usuarios	P95 menor a 2 s	235 lecturas; p95 296,36 ms; máximo 369,42 ms; 0 fallos	Aprobado
CP-25	RNF-02	Identidad, roles y transporte	OIDC/PKCE, HTTPS, HSTS, Argon2id y control de rol	CP-01 y CP-15–CP-19; sondas; Postman T4; HTTPS/HSTS verificados el 06/10	Aprobado
CP-26	RNF-03	Venta y estados en las vistas	Venta en hasta 3 pasos; 4 estados en 3 vistas	3 pasos y 12 de 12 estados	Aprobado
CP-27	RNF-04	Adaptación en navegadores y dispositivos	Sin desplazamiento a 1366 × 768 y 768 × 1024; Chrome y Edge; dos dispositivos	Pruebas automáticas a ambos tamaños; 16 capturas en Edge 154; Chrome 154.0.8037.98 a 1366 × 768 y 768 × 1024; APK 0.4.0 en Honor 50 con Android 13 y Huawei Y9s con Android 10	Aprobado

ID	Criterio	Escenario	Esperado	Obtenido	Estado
CP-28	RNF-05	Disponibilidad y aviso de canal caído	Al menos 99 % durante 7 días; aviso menor a 10 s	Corte formal: 100 % en 9 d 20 h hasta el 02/10, sin incidentes ni caída. Continuidad posterior: 17 d 11 h al 10/10, al 100 % y sin incidentes; <i>Last 30 days</i> no equivale a 30 días medidos. Aviso al segundo.	Aprobado
CP-29	CA-13.1	Rol autorizado agota producto o categoría	Deja de ofrecerse; nueva venta rechazada; propagación menor a 2 s	409 PRODUCTO_NO_DISPONIBLE; máximo 315 ms	Aprobado
CP-30	CA-13.2	Se agota un producto ya incluido	El pedido existente no cambia	2 líneas; Bs 70; estado pendiente	Aprobado
CP-31	D-76	Un rol intenta modificar la categoría del otro	403 sin cambios	403 ROL_SIN_PERMISO; datos sin modificación	Aprobado

Tabla F.10. Matriz completa CP-01 a CP-31. Fuente: docs/pruebas/README.md y reportes vigentes al 08/10/2026.

F.12 Evidencia complementaria

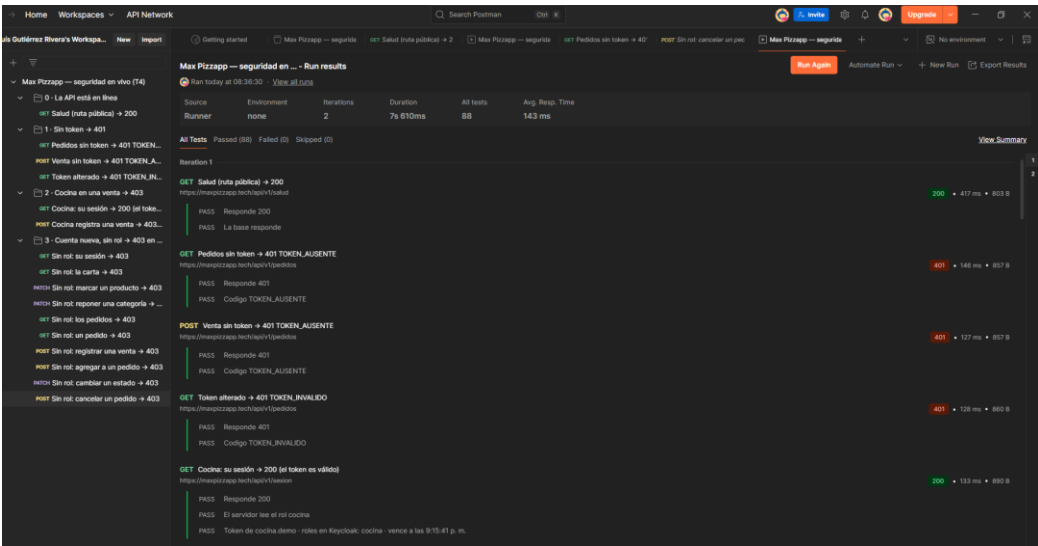


Figura F.2. Ejecución de comprobaciones de seguridad contra producción. Fuente: captura propia.

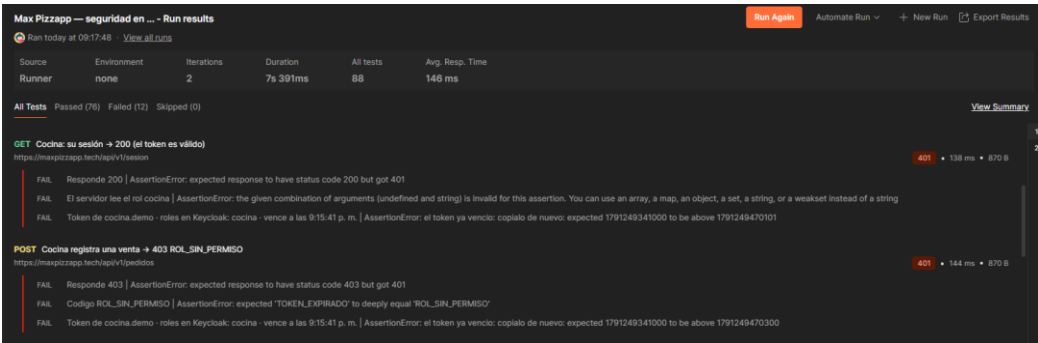


Figura F.3. Respuesta observada después del vencimiento del token. Fuente: captura propia.

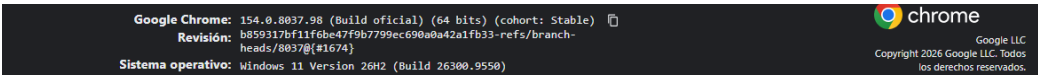


Figura F.4. Versión de Chrome identificada para CP-27. Fuente: captura propia.

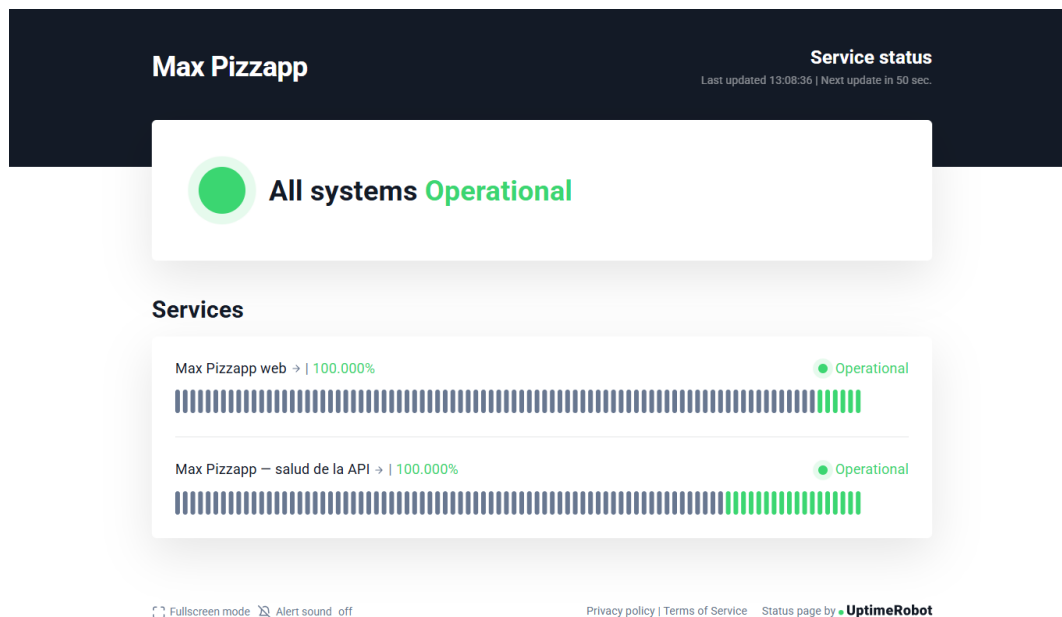


Figura F.5. Página pública de estado del monitoreo. Fuente: UptimeRobot, 10/10/2026.